context musings

hans hagen

Content

Introduction 4

1 Children of TEX 6

2 Advertising TEX 32

3 Why use TEX? 36

4 What's to stay, what's to go 44

5 Stability 52

6 METATEX, a roadmap 60

7 About what CONTEXT isn't 68

Introduction

This is a collection of articles and wrap-ups that don't suit in other manuals or collections. Some are published, some meant as draft for a presentation.

The "Children of TEX" article is the framework for a presentation at BachoTEX 2017 in Poland, and covers the main theme of the conference. In the aftermath of that conference I wrote "Advertising TEX" and later "Why use TEX?". The 2018 BachoTEX conference theme is explored in "What's to stay, what's to go". After a short discussion on the CONTEXT mailing list about stability (at the moment that MKII had been frozen for more than a decade but is still used without problems) I wrote "Stability".

Many of the thoughts in these articles are influenced by discussions with my colleagues Ton Otten and Kees van Marle. Operating in a similar arena, they provide me the reflection needed to sort out my thoughts on these matters.

Hans Hagen
Hasselt NL
2017–1028

1 Children of TEX

1.1 The theme

Nearly always TEX conferences carry a theme. As there have been many conferences the organizers have run out of themes involving fonts, macros and typesetting and are now cooking up more fuzzy ones. Take the BachoTUG 2017 theme:

Premises The starting point, what we have, what do we use, what has been achieved?
Predilections How do we act now, how do we want to act, what is important to us and what do we miss?
Predictions What is the future of TEX, what we'd like to achieve and can we influence it?

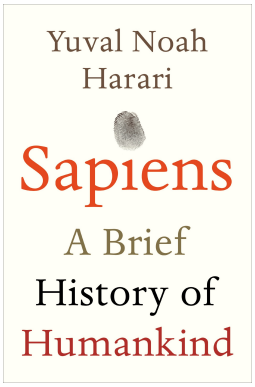
My first impression with these three P words was: what do they mean? Followed by the thought: this is no longer a place to take kids to. But the Internet gives access to the Cambridge Dictionary, so instead of running to the dusty meter of dictionaries somewhere else in my place, I made sure that I googled the most recent definitions:

premise an idea or theory on which a statement or action is based
predilection if someone has a predilection for something, they like it a lot
prediction a statement about what you think will happen in the future

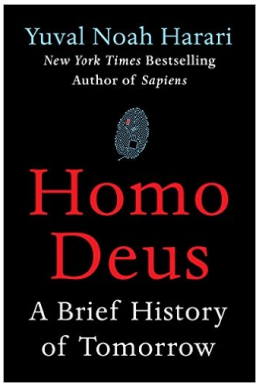
I won't try to relate these two sets of definitions but several words stand out in the second set: idea, theory, action, like, statement and future. Now, as a preparation for the usual sobering thoughts that Jerzy, Volker and I have when we staring into a BachoTEX campfire I decided to wrap up some ideas around these themes and words. The books that I will mention are just a selection of what you can find distributed around my place. This is not some systematic research but just the result of a few weeks making a couple of notes while pondering about this conference.

1.2 Introduction

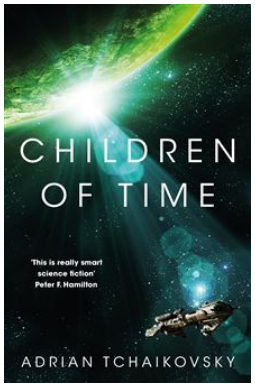
One cannot write the amount of TEX macros that I've written without also liking books. If you look at my bookshelves the topics are somewhat spread over the possible spectrum of topics: history, biology, astronomy, paleontology, general science but surprisingly little math. There are a bunch of typography-related books but only some have been read: it's the visuals that matter most and as there are no real developments I haven't bought new ones in over a decade, although I do buy books that look nice for our office display but the content should be interesting too. Of course I do have a couple of books about computer (related) science and technology but only a few are worth a second look. Sometimes I bought computer books expecting to use them (in some



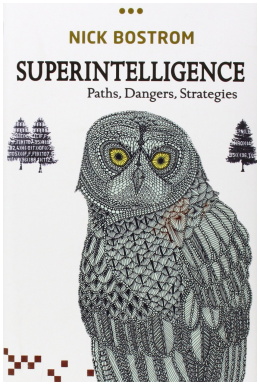
history



futurology



science fiction



informatics

Figure 1.1

project) but I must admit that most have not been read and many will soon end up in the paper bin (some already went that way). I'll make an exception for Knuth, Wirth and a few other fundamental ones that I (want to) read. And, I need to catch up on deep learning, so that might need a book.

My colleagues and I have many discussions, especially about what we read, and after a few decades one starts seeing patterns. Therefore the last few years it was a pleasant surprise for me to run into books and lectures that nicely summarize what one has noticed and discussed in a consistent way. My memory is not that good, but good enough to let some bells ring.

The first book that gave me this "finally a perfect summary of historic developments" feeling is "Sapiens" by Yuval Noah Harari. The author summarizes human history from a broad perspective where modern views on psychology, anthropology and technical developments are integrated. It's a follow up on a history writing trend started by Jared Diamond. The follow up "Homo Deus" looks ahead and is just as well written. It also integrates ideas from other fields, for instance those related to development of artificial intelligence (Dennett, Bostrom, etc.).

Another inspiration for this talk and article is the 50 hour lecture series on behavioral biology by Robert Sapolsky of Stanford University, brought to my attention by my nephew Bram who visited a few TEX conferences with me and who is now also forced to use TEX for assignments and reports. (How come self-published books used at universities often look so bad?)

The title of this talk is inspired by the book "Children of Time" by Adrian Tchaikovsky that I read recently. There are science fiction writers who focus on long term science and technology, such as some of Alastair Reynolds, while others follow up on recent development in all kind of sciences. One can recognize aspects of "Superintelligence" by Bostrom in Neal Asher's books, insights in psychology in the older Greg Bear books, while in the mentioned "Children of Time" (socio)biological insights dominate. The main thread in that book is the development of intelligence, social behaviour, language, script and cooperation in a species quite different from us: spiders. It definitely avoids the anthropocentric focus that we normally have.

So how does this relate to the themes of the BachoTEX conference? I will pick out some ways to approach them using ideas from the kind of resources mentioned above. I could probably go on and on for pages because once you start relating what you read and hear to this TEX ecosystem and community, there is no end. So, consider this a snapshot, that somehow relates to the themes:

premise Let's look at what the live sciences have to say about TEX and friends and let's hope that I don't offend the reader and the field.

predilection Let's figure out what brings us here to this place deeply hidden in the woods, a secret gathering of the TEX sect.

prediction Let's see if the brains present here can predict the future because after all, according to Dennett, that is what brains are for.

At school I was already intrigued by patterns in history: a cyclic, spiral and sinusoid social evolution instead of a pure linear sequence of events. It became my first typeset-by-typewriter document: Is history an exact science? Next I will use and abuse patterns and ideas to describe the TEX world, not wearing a layman's mathematical glasses, but more from the perspective of live sciences, where chaos dominates.

1.3 The larger picture

History of mankind can be roughly summarized as follows. For a really long time we were hunters but at some point (10K years ago) became farmers. As a result we could live in larger groups and still feed them. The growing complexity of society triggered rules and religion as instruments for stability and organization (I use the term religion in its broadest sense here). For quite a while cultures came and went, and climate changes are among the reasons.

After the industrial revolution new religions were invented (social, economic and national liberalism) and we're now getting dataism (search for Harari on youtube for a better summary). Some pretty great minds seem to agree that we're heading to a time when humans as we are will be outdated. Massive automation, interaction between the self and computer driven ecosystems, lack of jobs and purpose, messing around with our genome. Some countries and cultures still have to catch up on the industrial revolution, if they manage at all, and maybe we ourselves will be just as behind reality soon. Just ask yourself: did you manage to catch up? Is TEX a stone age tool or a revolutionary turning point?

A few decades ago a trip to BachoTEX took more than a day. Now you drive there in just over half a day. There was a time that it took weeks: preparation, changing horses, avoiding bad roads. Not only your own man-hours were involved. It became easier later (my first trip took only 24 hours) and recently it turned into a piece of cake: you don't pick up maps but start your device; you don't need a travel agent but use the Internet;

there are no border patrols, you can just drive on. (Okay, maybe some day soon border patrols at the Polish border show up again, just like road tax police in Germany, but that might be a temporary glitch.)

Life gets easier and jobs get lost. Taxi and truck drivers, travel agents, and cashiers become as obsolete as agricultural workers before. Next in line are doctors, lawyers, typesetters, printers, and all those who think they're safe. Well, how many people were needed 400 years ago to produce the proceedings of a conference like this in a few days' time span? Why read the introduction of a book or a review when you can just listen to the author's summary on the web? How many conferences still make proceedings (or go for videos instead), will we actually need editors and typesetters in the future? How much easier has it become to design a font, including variants? What stories can designers tell in the future when programs do the lot? The narrower your speciality is, the worse are your changes; hopefully the people present at this conference operate on a broader spectrum. It's a snapshot. I will show some book covers as reference but am aware that years ago or ahead the selection could have been different.

1.4 Words

Words (whatever they represent) found a perfect spot to survive: our minds. Then they made it from speech (and imagination) into writing: carved in stone, wood, lead. At some point they managed to travel over wires but no matter what happened, they are still around. Typesetting as visualization is also still surrounding us so that might give us a starting point for ensuring a future for TEX to work on, because TEX is all about words. There is a lot we don't see; imagine if our eyes had microscopic qualities. What if we could hear beyond 20KHz. Imagine we could see infrared. How is that with words. What tools, similar in impact as TEX, can evolve once we figure that out. What if we get access to the areas of our brain that hold information? We went from print to screen and TEX could cope with that. Can it cope with what comes next?

The first printing press replaced literal copying by hand. Later we got these linotype-like machines but apart from a few left, these are already thrown out of windows (as we saw in a movie a few BachoTEX's ago). Photo-typesetting has been replaced too and because a traditional centuries old printing press is a nice to see item, these probably ring more bells than that gray metal closed box typesetters. Organizers of TEX conferences love to bring the audience to old printing workshops and museums. At some point computers got used for typesetting and in that arena TEX found its place. These gray closed boxes are way less interesting than something mechanical that at least invites us to touch it. How excited can one be about a stack of TEX Live DVDs?

1.5 Remembering

Two times I visited the part of the science museum in London with young family members: distracted by constantly swiping their small powerful devices, they didn't have the least interest in the exhibited computer related items, let alone the fact that the couch

they were sitting on was a Cray mainframe. Later on, climbing on some old monument or an old cannon seemed more fun. So, in a few decades folks will still look at wooden printing presses but quickly walk through the part of an exhibition where the tools that we use are shown. We need to find ways to look interesting. But don't think we're unique: how many kids find graphical trend-setting games like Myst and Riven still interesting? On the other hand a couple of month ago a bunch of nieces and nephews had a lot of fun with an old Atari console running low-res bitmap games. Maybe there is hope for good old TEX.

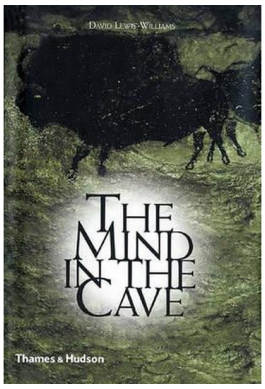
If indeed we're heading to a radically different society one can argue if this whole discussion makes sense. When the steam engine showed up, the metaphor for what went on in our heads was that technology, It's a popular example of speakers on this topic: "venting off steam". When electricity and radio came around metaphors like "being on the same wavelength" showed up. A few decades ago the computer replaced that model although in the meantime the model is more neurobiological: we're a hormone and neurotransmitter driven computer. We don't have memory the way computers do.

How relevant will page breaks, paragraph and line breaks be in the future? Just like "venting off steam" may make no sense to the youth, asking a typesetter to "give me a break" might not make much sense soon. However, when discussing automated typesetting the question "are we on the same page" still has relevance.

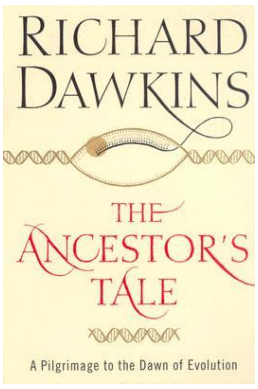
Typesetting with a computer might seem like the ultimate solution but it's actually rather dumb when we consider truly intelligent systems. On the large scale of history and developments what we do might get quite unnoticed. Say that mankind survives the next few hundred years one way or the other. Science fiction novels by Jack McDevitt have an interesting perspective of rather normal humans millennia ahead of us who look back on these times in the same way as we look back now. Nothing fundamental changed in the way we run society. Nearly nothing from the past is left over and apart from being ruled by AIs people still do sort of what they do now. TEX? What is that? Well, there once was this great computer scientist Knuth (in the remembered row of names like Aristotle —I just started reading "The Lagoon" by Armand Leroi— Newton, Einstein, his will show up) who had a group of followers that used a program that he seems to have written. And even that is unlikely to be remembered, unless maybe user groups manage to organize an archive and pass that on. Maybe the fact that TEX was one of the first large scale open source programs, of which someone can study the history, makes it a survivor. The first program that was properly documented in detail! But then we need to make sure that it gets known and persists.

1.6 Automation

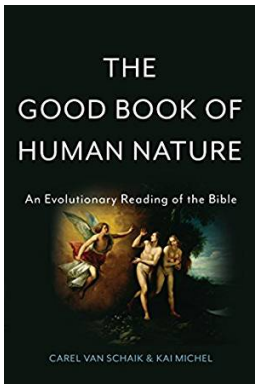
In a recent interview Daniel Dennett explains that his view of the mind as a big neural network, one that can be simulated in software on silicon, is a bit too simplistic. He wonders if we shouldn't more tend to think of a network of (selfish) neurons that group together in tasks and then compete with each other, if only because they want to have something to do.



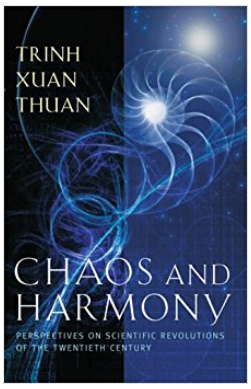
paleontology



evolutionary biology



anthropology



physics

Figure 1.2

Maybe attempts to catch the creative mindset and working of a typesetter in algorithms is futile. What actually is great typography or good typesetting? Recently I took a look at my bookshelf wondering what to get rid of — better do that now than when I'm too old to carry the crap down (crap being defined as uninteresting content or bad looking). I was surprised about the on-the-average bad quality of the typesetting and print. It's also not really getting better. One just gets accustomed to what is the norm at a certain point. Whenever they change the layout and look and feel of the newspaper I read the arguments are readability and ease of access. Well, I never had such a hard time reading my paper as today (with my old eyes).

Are we, like Dennett, willing to discard old views on our tools and models? When my first computer was a RCA 1802 based kit, that had 256 bytes of memory. My current laptop (from 2013) is a Dell Precision workstation with an extreme quad core processor and 16 GB of memory and ssd storage. Before I arrived there I worked with DEC-10, VAX and the whole range of Intel CPUs. So if you really want to compare a brain with a computer, take your choice.

I started with TEX on a 4 MHz desk top with 640 MB memory and a 10 MB hard disk. Running CONTEXT MKIV with LUATEX on such a machine is no option at all, but I still carry the burden of trying to write efficient code (which is still somewhat reflected in the code that makes up CONTEXT). In the decades that we have been using TEX we had to adapt! Demands changed, possibilities changed, technologies changed. And they keep changing. How many successive changes can a TEX user handle? Sometimes, when I look and listen I wonder.

If you look back, that is, if you read about the tens of thousands of years that it took humans to evolve ("The mind in the cave" by Lewis-Williams is a good exercise) you realize even more in what a fast-paced time we live and that we're witnessing transitions of another magnitude.

In the evolution of species some tools were invented multiple times, like eyes. You see the same in our TEX world: multiple (sub)macro packages, different font technologies, the same solutions but with an alternative approach. Some disappear, some

stay around. Just like different circumstances demand different solutions in nature, so do different situations in typesetting, for instance different table rendering solutions. Sometime I get the feeling that we focus too much on getting rid of all but one solution while more natural would be to accept diversity, like bio-diversity is accepted. Transitions nowadays happen faster but the question is if, like aeons before, we (have to) let them fade away. When evolution is discussed the terms 'random', 'selection', 'fit', and so on are used. This probably also applies to typography: at some point a font can be used a lot, but in the end the best readable and most attractive one will survive. Newspapers are printed in many copies, but rare beautiful books hold value. Of course, just like in nature some developments force the further path of development, we don't suddenly grow more legs or digits on our hands. The same happens with TEX on a smaller timescale: successors still have the same core technology, also because if we'd drop it, it would be something different and then give a reason to reconsider using such technology (which likely would result in going by another path).

1.7 Quality

Richard Dawkins "The Ancestor's Tale" is a non-stop read. In a discussion with Jared Diamond about religion and evolution they ponder this thread: you holding the hand of your mother who is handing her mother's hand and so on till at some point fish get into the picture. The question then is, when do we start calling something human? And a related question is, when does what we call morality creeps in? Is 50% neanderthaler human or not?

So, in the history of putting thoughts on paper: where does TEX fit in? When do we start calling something automated typesetting? When do we decide that we have quality? Is TEX so much different from its predecessors? And when we see aspects of TEX (or related font technology) in more modern programs, do we see points where we cross qualitative or other boundaries? Is a program doing a better job than a human? Where do we stand? There are fields where there is no doubt that machines outperform humans. It's probably a bit more difficult in aesthetic fields except perhaps when we lower the conditions and expectations (something that happens a lot).

For sure TEX will become obsolete, maybe even faster that we think, but so will other typesetting technologies. Just look back and have no illusions. Till then we can have our fun and eventually, when we have more free time than we need, we might use it out of hobbyism. Maybe TEX will be remembered by probably its most important side effect: the first large scale open source, the time when users met over programs, Knuth's disciples gathered in user groups, etc. The tools that we use are just a step in an evolution. And, as with evolution, most branches are pruned. So, when in the far future one looks back, will they even notice TEX? The ancestor's tail turns the tree upside down: at the end of the successful branch one doesn't see the dead ends.

Just a thought: CDs and media servers are recently being replaced (or at least accompanied) by Long Play records. In the shop where I buy my CDs the space allocated to

records grows at the cost of more modern media. So, maybe at some point retro-typesetting will pop up. Of course it might skip TEX and end up at woodcutting or printing with lead.

1.8 What mission

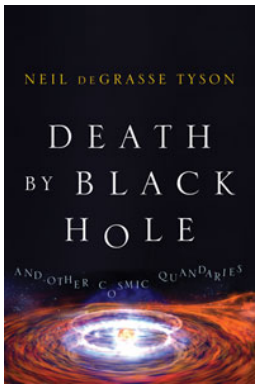
We rely on search engines instead of asking around or browsing libraries. Do students really still read books and manuals or do they just search and listen to lectures. Harari claims that instead of teaching kids facts in school we should just take for granted that they can get all the data they want and that we should learn them how to deal with data and adapt to what is coming. We take for granted that small devices with human voices show us the route to drive to BachoTEX, for instance, although by now I can drive it without help. In fact, kids can surprise you by asking if we're driving in Germany when we are already in Poland.

We accept that computer programs help physicians in analyzing pictures. Some wear watches that warn them about health issues, and I know a few people who monitor their sugar levels electronically instead of relying on their own measurements. We seem to believe and trust the programs. And indeed, we also believe that TEX does the job in the best way possible. How many people really understand the way TEX works?

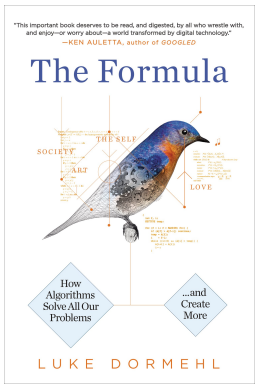
We still have mailing lists where we help each other. There are also wikis and forums like stack exchange. But who says that even a moderate bit of artificial intelligence doesn't answer questions better. Of course there needs to be input (manuals, previous answers, etc.) but just like we need fewer people as workforce soon, the number of experts needed also can be smaller. And we're still talking about a traditional system like TEX. Maybe the social experience that we have on these media will survive somehow, although: how many people are members of societies, participate in demonstrations, meet weekly in places where ideas get exchanged, compared to a few decades ago? That being said, I love to watch posts with beautiful CONTEXT solutions or listen to talks by enthusiastic users who do things I hadn't expected. I really hope that this property survives, just like I hope that we will be able to see the difference between a real user's response and one from an intelligent machine (an unrealistic hope I fear). Satisfaction wins and just like our neurological subsystems at some point permanently adapt to thresholds (given that you trigger things often enough), we get accustomed to what TEX provides and so we stick to it.

1.9 Intelligence versus consciousness

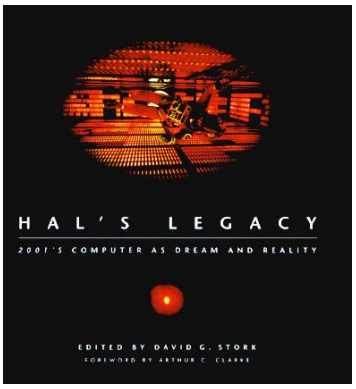
Much of what we do is automated. You don't need to think of which leg to move and what foot to put down when you walk. Reacting to danger also to a large extent is automated. It doesn't help much to start thinking about how dangerous a lion can be when it's coming after you, you'd better move fast. Our limbic system is responsible for such automated behaviour, for instance driven by emotions. The more difficult tasks and thoughts about them happen in the frontal cortex (sort of).



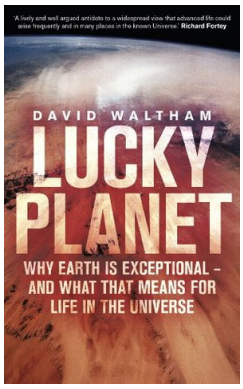
astronomy



informatics



future science



earth science

Figure 1.3

For most users TEX is like the limbic system: there is not much thinking involved, and the easy solutions are the ones used. Just like hitting a nerve triggers a chain of reactions, hitting a key eventually produces a typeset document. Often this is best because the job needs to get done and no one really cares how it looks; just copy a preamble, key in the text and assume that it works out well (enough). It is tempting to compare TEX's penalties, badness and other parameters with levels of hormones and neurotransmitters. Their function depends on where they get used and the impact can be accumulated, blocked or absent. It's all magic, especially when things interact.

Existing TEX users, developers and user groups of course prefer to think otherwise, that it is a positive choice by free will. That new users have looked around and arrived at TEX for good reason: their frontal cortex steering a deliberate choice. Well, it might have played a role but the decision to use TEX might in the end be due to survival skills: I want to pass this exam and therefore I will use that weird system called TEX.

All animals, us included, have some level of intelligence but also have this hard to describe property that we think makes us what we are. Intelligence and consciousness are not the same (at least we know a bit about the first but nearly nothing about the second). We can argue about how well composed some music is but why we like it is a different matter.

We can make a well thought out choice for using TEX for certain tasks but can we say why we started liking it (or not)? Why it gives us pleasure or maybe grief? Has it become a drug that we got addicted to? So, one can make an intelligent decision about using TEX but getting a grip on why we like it can be hard. Do we enjoy the first time struggle? Probably not. Do we like the folks involved? Yes, Don Knuth is a special and very nice person. Can we find help and run into a friendly community? Yes, and a unique one too, annoying at times, often stimulating and on the average friendly for all the odd cases running around.

Artificial intelligence is pretty ambitious, so speaking of machine intelligence is probably better. Is TEX an intelligent program? There is definitely some intelligence built in and the designer of that program is for sure very intelligent. The designer is also a conscious entity: he likes what he did and finds pleasure in using it. The program on

the other hand is just doing its job: it doesn't care how it's done and how long it takes: a mindless entity. So here is a question: do we really want a more intelligent program doing the job for us, or do those who attend conferences like BachoTEX enjoy TEXing so much that they happily stay with what they have now? Compared to rockets tumbling down and/or exploding or Mars landers thrashing themselves due to programming errors of interactions, TEX is surprisingly stable and bug free.

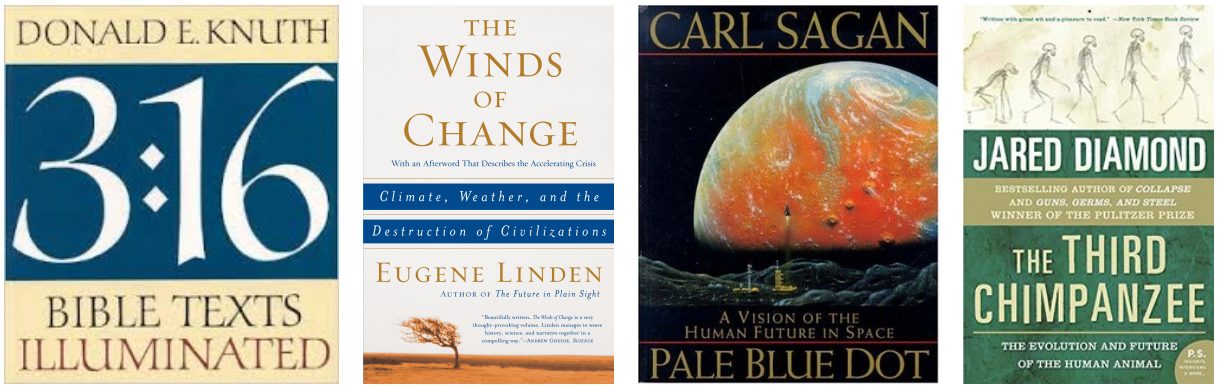
1.10 Individual versus group evolution

After listening for hours to Sapolsky you start getting accustomed to remarks about (unconscious) behaviour driven by genes, expression and environment, aimed at "spreading many copies of your genes". In most cases that is an individual's driving force. However, cooperation between individuals plays a role in this. A possible view is that we have now reached a state where survival is more dependent on a group than on an individual. This makes sense when we consider that developments (around us) can go way faster than regular evolution (adaptation) can handle. We take control over evolution, a mechanism that needs time to adapt and time is something we don't give it anymore.

Why does TEX stay around? It started with an individual but eventually it's the groups that keeps it going. A too-small group won't work but too-large groups won't work either. It's a known fact that one can only handle some 150 social contacts: we evolved in small bands that split when they became too large. Larger groups demanded abstract beliefs and systems to deal with the numbers: housing, food production, protection. The TEX user groups also provide some organization: they organize meetings, somehow keep development going and provide infrastructure and distributions. They are organized around languages. According to Diamond new languages are still discovered but many go extinct too. So the potential for language related user groups is not really growing.

Some of the problems that we face in this world have become too large to be dealt with by individuals and nations. In spite of what anti-globalists want we cannot deal with our energy hunger, environmental issues, lack of natural resources, upcoming technologies without global cooperation. We currently see a regression in cooperation by nationalistic movements, protectionism and the usual going back to presumed better times, but that won't work.

Local user groups are important but the number of members is not growing. There is some cooperation between groups but eventually we might need to combine the groups into one which might succeed unless one wants to come first. Of course we will get the same sentiments and arguments as in regular politics but on the other hand, we already have the advantage of TEX systems being multi-lingual and users sharing interest in the diversity of usage and users. The biggest challenge is to pass on what we have achieved. We're just a momentary highlight and let's not try to embrace some "TEX first" madness.



art history astronomy history

Figure 1.4

1.11 Sexes

Most species have two sexes but it is actually a continuum controlled by hormones and genetic expression: we just have to accept it. Although the situation has improved there are plenty of places where some gender relationships are considered bad even to the extent that one's life can be in danger. Actually having strong ideas about these issues is typically human. But in the end one has to accept the continuum.

In a similar way we just have to accept that TEX usage, application of TEX engines, etc. is a continuum and not a batch versus WYSIWYG battle any more. It's disturbing to read strong recommendations not to use this or that. Of the many macro packages that showed up only a few were able to survive. How do users of outlines look at bitmaps, how do DVI lovers look at PDF. But, as typesetting relates to esthetics, strong opinions come with the game.

Sapolsky reports about a group of baboons where due to the fact that they get the first choice of food the alpha males of pack got poisoned, so that the remaining suppressed males who treated the females well became dominant. In fact they can then make sure that no new alpha male from outside joins the pack without behaving like they do. A sort of social selection. In a similar fashion, until now the gatherings of TEXies managed to keep its social properties and has not been dominated by for instance commerce.

In the animal world often sexes relate to appearance. The word sexy made it to other domains as well. Is TEX sexy? For some it is. We often don't see the real colors of birds. What looks gray to us looks vivid to a bird which sees in a different spectrum. The same is true for TEX. Some users see a command line (shell) and think: this is great! Others just see characters and keystrokes and are more attracted to an interactive program. When I see a graphic made by METAPOST, I always note how exact it is. Others don't care if their interactive effort doesn't connect the dots well. Some people (also present here) think that we should make TEX attractive but keep in mind that like and dislike are not fixed human properties. Some mindsets might as well be the result from our makeup, others can be driven by culture.

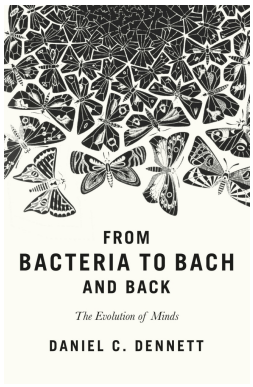
1.12 Religion

One of Sapolsky's lectures is about religion and it comes in the sequence of mental variations including depression and schizophrenia, because all these relate to mental states, emotions, thresholds and such (all things human). That makes it a tricky topic which is why it has not been taped. As I was raised in a moderate Protestant tradition I can imagine that it's an uncomfortable topic instead. But there are actually a few years older videos around and they are interesting to watch and not as threatening as some might expect. Here I just stick to some common characteristics.

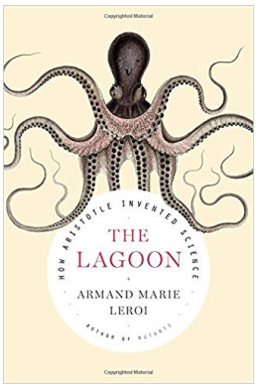
If you separate the functions that religions play into for instance explanation of the yet unknown, social interactions, control of power and regulation of morals, then it's clear why at TEX user group meetings the religious aspect of TEX has been discussed in talks. Those who see programs as infallible and always right and don't understand the inner working can see it as an almighty entity. In the Netherlands church-going diminishes but it looks like alternative meetings are replacing it (and I'm not talking of football matches). So what are our TEX meetings? What do we believe in? The reason that I bring up this aspect is that in the TEX community we can find aspects of the more extremist aspects of religions: if you don't use the macro package that I use, you're wrong. If you don't use the same operating system as I do, you're evil. You will be punished if you use the wrong editor for TEX? Why don't you use this library (which, by the way, just replaced that other one)? We create angels and daemons. Even for quite convinced atheists (it's not hard to run into them on youtube) a religion only survives when it has benefits, something that puzzles them. So when we're religious about TEX and friends we have to make sure that it's at least beneficial. Also, maybe we fall in Dennett's category of "believers who want to believe": it helps us to do our job if we just believe that we have the perfect tool. Religion has inspired visual and aural art and keeps doing that. (Don Knuth's current musical composition project is a good example of this.)

Scientists can be religious, in flexible ways too, which is demonstrated by Don Knuth. In fact, I'm pretty sure TEX would not be in the position it is in now if it weren't for his knowledgeable, inspirational, humorous, humble, and always positive presence. And for sure he's not at all religious about the open source software that he sent viral.

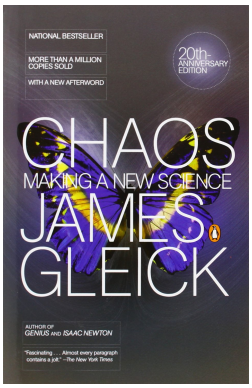
I'm halfway through reading "The Good Book of Human Nature" (An Evolutionary Reading of the Bible) a book about the evolution of the bible and monotheism which is quite interesting. It discusses for instance how transitions from a hunter to a farmer society demanded a change of rules and introduced stories that made sense in that changing paradigm. Staying in one place means that possessions became more important and therefore inheritance. Often when religion is discussed by behavioral biologists, historians and anthropologists they stress this cultural narrative aspect. Also mentioned is that such societies were willing to support (in food and shelter) the ones that didn't normally fit it but added to the spiritual character of religions. The social and welcoming aspect is definitely present in for instance BachoTEX conferences although a bystander



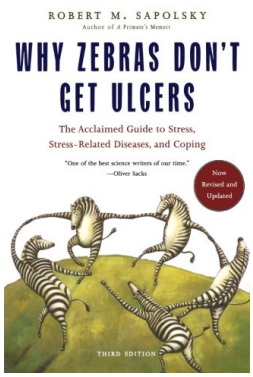
philosophy



science history



science



behavioral biology

Figure 1.5

can wonder what these folks are doing in the middle of the night around a campfire, singing, drinking, frying sausages, spitting fire, and discussing the meaning of life.

Those who wrap up the state of religious affairs, do predictions and advocate the message, are sometimes called evangelists. I remember a TEX conference in the USA where the gospel of XML was preached (by someone from outside the TEX community). We were all invited to believe it. I was sitting in the back of the crowded (!) room and that speaker was not at all interested in who spoke before and after. Well, I do my share of XML processing with CONTEXT, but believe me: much of the XML that we see is not according to any gospel. It's probably blessed the same way as those state officials get blessed when they ask and pray for it in public.

It can get worse at TEX conferences. Some present here at BachoTEX might remember the PDF evangelists that we had show up at TEX conferences. You see this qualification occasionally and I have become quite allergic to qualifications like architect, innovator, visionary, inspirator and evangelist, even worse when they look young but qualify as senior. I have no problem with religion at all but let's stay away from becoming one. And yes, typography also falls into that trap, so we have to be doubly careful.

1.13 Chaotic solutions

The lectures on "chaos and reductionism" and "emergence and complexity" were the highlights in Sapolsky's lectures. I'm not a good narrator so I will not summarize them but it sort of boils down to the fact that certain classes of problems cannot be split up in smaller tasks that we understand well, after which we can reassemble the solutions to deal with the complex task. Emerging systems can however cook up working solutions from random events. Examples are colonies of ants and bees.

The TEX community is like a colony: we cook up solutions, often by trial and error. We dream of the perfect solutions but deep down know that esthetics cannot be programmed in detail. This is a good thing because it doesn't render us obsolete. At last year's BachoTEX, my nephew Teun and I challenged the anthill outside the canteen to

typeset the TEX logo with sticks but it didn't persist. So we don't need to worry about competition from that end. How do you program a hive mind anyway?

When chaos theory evolved in the second half of the previous century not every scientist felt happy about it. Instead of converging to more perfect predictions and control in some fields a persistent uncertainty became reality.

After about a decade of using TEX and writing macros to solve recurring situations I came to the conclusion that striving for a perfect TEX (the engine) that can do everything and anything makes no sense. Don Knuth not only stopped adding code when he could do what he needed for his books, he also stuck to what to me seems reasonable endpoints. Every hard-coded solution beyond that is just that: a hard-coded solution that is not able to deal with the exceptions that make up most of the more complex documents. Of course we can theorize and discuss at length the perfect never-reachable solutions but sometimes it makes more sense to admit that an able user of a desktop publishing system can do that job in minutes, just by looking at the result and moving around an image or piece of text a bit.

There are some hard-coded solutions and presets in the programs but with LUATEX and MPLIB we try to open those up. And that's about it. Thinking that for instance adding features like protrusion or expansion (or whatever else) always lead to better results is just a dream. Just as a butterfly flapping its wings on one side of the world can have an effect on the other side, so can adding a single syllable to your source completely confuse an otherwise clever column or page break algorithm. So, we settle for not adding more to the engine, and provide just a flexible framework.

A curious observation is that when Edward Lorenz ran into chaotic models it was partially due to a restart of a simulation midway, using printed floating point numbers that then in the computer were represented with a different accuracy than printed. Aware of floating point numbers being represented differently across architectures, Don Knuth made sure that TEX was insensitive to this so that its outcome was predictable, if you knew how it worked internally. Maybe LUATEX introduces a bit of chaos because the LUA we use has only floats. In fact, a few months ago we did uncover a bug in the backend where the same phenomena gave a chaotic crash.

In chaos theory there is the concept of an attractor. When visualized this can be the area (seemingly random) covered by a trajectory. Or it can be a single point where for instance a pendulum comes to rest. So what is our attractor? We have a few actually. First there is the engine, the stable core of primitives always present. You often see programs grow more complex every update and for sure that happened with ε-TEX, PDFTEX, XETEX and LUATEX. However there is always the core that is supposed to be stable. After some time the new kid arrives at a stable state not much different from the parent. The same is true for METAPOST. Fonts are somewhat different because the technology changes but in the end the shapes and their interactions become stable as well. Yet another example is TEX Live: during a year it might diverge from its route but eventually it settles down and enters the area where we expect it to end up. The TEX world is at times chaotic, but stable in the long run.

So, how about the existence, the reason for it still being around? One can speculate about its future trajectory but one thing is sure: as long as we break a text into paragraphs and pages TEX is hard to beat. But what if we don't need that any more? What if the concept of a page is no longer relevant? What if justified texts no longer matter (often designers don't care anyway)? What if students are no longer challenged to come up with a nice looking thesis? Do these collaborative tools with remote TEX processing really bring new long term users or is TEX then just one of the come-and-go tools?

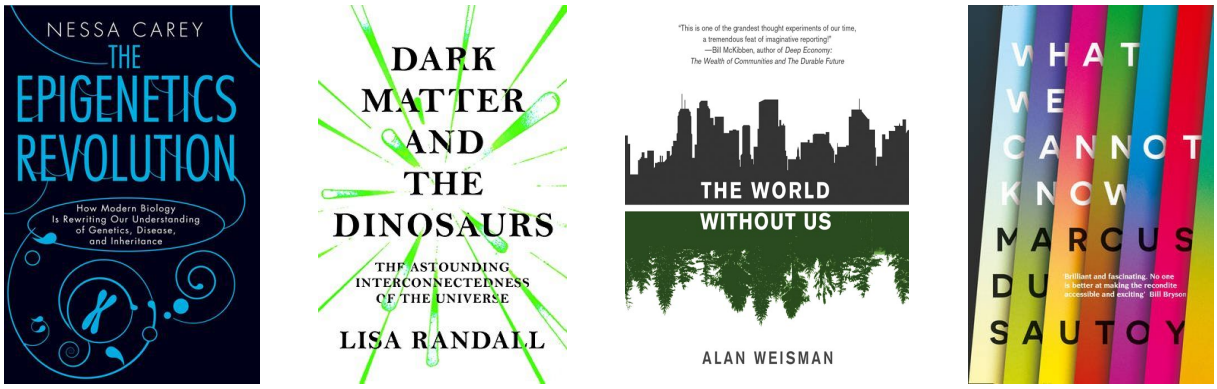
1.14 Looking ahead

In an interview ("World of ideas") Asimov explains that science fiction evolved rapidly when people lived long enough to see that there was a future (even for their offspring) that is different from today. It is (at least for me) mind boggling to think of an evolution of hundreds of thousands of years to achieve something like language. Waiting for the physical being to arrive at a spot where you can make sounds, where the brain is suitable for linguistic patterns, etc. A few hundred years ago speed of any developments (and science) stepped up.

TEX is getting near 40 years old. Now, for software that **is** old! In that period we have seen computers evolve: thousands of times faster processing, even more increase in memory and storage. If we read about spaceships that travel at a reasonable fraction of the speed of light, and think that will not happen soon, just think back to the terminals that were sitting in computer labs when TEX was developed: 300 baud was normal. I actually spent quite some time on optimizing time-critical components of CONTEXT but on this timescale that is really a waste of time. But even temporary bottlenecks can be annoying (and costly) enough to trigger such an effort. (Okay, I admit that it can be a challenge, a kind of game, too.)

Neil Tyson, in the video "Storytelling of science" says that when science made it possible to make photos it also made possible a transition in painting to impressionism. Other technology could make the exact snapshot so there was new room for inner feelings and impressions. When the Internet showed up we went through a similar transition, but TEX actually dates from before the Internet. Did we also have a shift in typesetting? To some extent yes, browsers and real time rendering is different from rendering pages on paper. In what space and time are TEXies rooted?

We get older than previous generations. Quoting Sapolsky "... we are now living well enough and long enough to slowly fall apart." The opposite is happening with our tools, especially software: it's useful lifetime becomes shorter and changes faster each year. Just look at the version numbers of operating systems. Don Knuth expected TEX to last for a long time and compared to other software its core concept and implementation is doing surprisingly well. We use a tool that suits our lifespan! Let's not stress ourselves out too much with complex themes. (It helps to read "Why zebras don't get ulcers".)



genetics physics history science

Figure 1.6

1.15 Memes

If you repeat a message often enough, even if it's something not true, it can become a meme that gets itself transferred across generations. Conferences like this is where they can evolve. We tell ourselves and the audience how good TEX is and because we spend so many hours, days, weeks, months using it, it actually must be good, or otherwise we would not come here and talk about it. We're not so stupid as to spend time on something not good, are we? We're always surprised when we run into a (potential) customer who seems to know TEX. It rings a bell, and it being around must mean something. Somehow the TEX meme has anchored itself when someone attended university. Even if experiences might have been bad or usage was minimal. The meme that TEX is the best in math typesetting is a strong survivor.

There's a certain kind of person who tries to get away with their own deeds and decisions by pointing to "fake news" and accusations of "mainstream media" cheating on them. But to what extent are our stories true about how easy TEX macro packages are to use and how good their result? We have to make sure we spread the right memes. And the user groups are the guardians.

Maybe macro packages are like memes too. In the beginning there was a bunch but only some survived. It's about adaptation and evolution. Maybe competition was too fierce in the beginning. Like ecosystems, organisms and cellular processes in biology we can see the TEX ecosystem, users and usage, as a chaotic system. Solutions pop up, succeed, survive, lead to new ones. Some look similar and slightly different input can give hugely different outcomes. You cannot really look too far ahead and you cannot deduce the past from the present. Whenever something kicks it off its stable course, like the arrival of color, graphics, font technologies, PDF, XML, ebooks, the TEX ecosystem has to adapt and find its stable state again. The core technology has proven to be quite fit for the kind of adaptation needed. But still, do it wrong and you get amplified out of existence, don't do anything and the external factors also make you extinct. There is no denial that (in the computer domain) TEX is surprisingly stable and adaptive. It's also hard not to see how conservatism can lead to extinction.

1.16 Inspiration

I just took some ideas from different fields. I could have mentioned quantum biology, which tries to explain some unexplainable phenomena in living creatures. For instance how do birds navigate without visible and measurable clues. How do people arrive at TEX while we don't really advertise? Or I could mention epigenetics and explorations in junk DNA. It's not the bit of the genome that we thought that matters, but also the expression of the genes driven by other factors. Offspring not only gets genetic material passed but it can get presets. How can the TEX community pass on Knuth's legacy? Do we need to hide the message in subtle ways? Or how about the quest for dark matter? Does it really exist or do we want (need) it to exist? Does TEX really have that many users, or do we cheat by adding the users that are enforced during college but don't like it at all? There's enough inspiration for topics at TEX conferences, we just have to look around us.

1.17 Stability

I didn't go into technical aspects of TEX yet. I must admit that after decades of writing macros I've reached a point where I can safely say that there will never be perfect automated solutions for really complex documents. When books about neural networks show up I wondered if it could be applied (but I couldn't). When I ran into genetic algorithms I tried to understand its possible impact (but I never did). So I stuck to writing solutions for problems using visualization: the trial and error way. Of course, speaking of CONTEXT, I will adapt what is needed, and others can do that as well. Is there a new font technology? Fine, let's support it as it's no big deal, just a boring programming task. Does a user want a new mechanism? No problem, as solving a reduced subset of problems can be fun. But to think of TEX in a reductionist way, i.e. solving the small puzzles, and to expect the whole to work in tandem to solve a complex task is not trivial and maybe even impossible. It's a good thing actually, as it keeps us on edge. Also, CONTEXT was designed to help you with your own solutions: be creative.

I mentioned my nephew Bram. He has seen part of this crowd a few times, just like his brother and sister do now. He's into artificial intelligence now. In a few years I'll ask him how he sees the current state of TEX affairs. I might learn a few tricks in the process.

In "The world without us" Weisman explores how fast the world would be void of traces of humankind. A mere 10.000 years can be more than enough. Looking back, that's about the time hunters became farmers. So here's a challenge: say that we want an ant culture that evolves to the level of having archaeologists to know that we were here at BachoTEX ... what would we leave behind?

Sapolsky ends his series by stressing that we should accept and embrace individual differences. The person sitting next to you can have the same makeup but be just a bit more sensitive to depression or be the few percent with genes controlling schizophrenic behaviour. He stresses that knowing how things work or where things go wrong doesn't

mean that we should fix everything. So look at this room full of TEXies: we don't need to be all the same, use all the same, we don't need some dominance, we just need to accept and especially we need to understand that we can never fully understand (and solve) everything forever.

Predictions, one of the themes, can be hard. It's not true that science has the answer to everything. There will always be room for speculation and maybe we will always need metaphysics too. I just started to read "What we cannot know" by Sautoy. For sure those present here can not predict how TEX will go on and/or be remembered.

1.18 Children of TEX

I mentioned "Children of time". The author lets you see their spidery world through spider eyes and physiology. They have different possibilities (eyesight, smell) than we do and also different mental capabilities. They evolve rapidly and have to cope conceptually with signals from a human surveillance satellite up in the sky. Eventually they need to deal with a bunch of (of course) quarrelling humans who want their place on the planet. We humans have some pre-occupation with spiders and other creatures. In a competitive world it is sometimes better to be suspicious (and avoid and flee) that to take a risk of being eaten. A frequently used example is that a rustle in a bush can be the wind or a lion, so best is to run.

We are not that well adapted to our current environment. We evolved at a very slow pace so there was no need to look ahead more than a year. And so we still don't look too far ahead (and choose politicians accordingly). We can also not deal that well with statistics (Dawkins's "Climbing Mount Probability" is a good read) so we make false assumptions, or just forget.

Does our typeset text really look that good on the long run, or do we cheat with statistics? It's not too hard to find a bad example of something not made by TEX and extrapolate that to the whole body of typeset documents. Just like we can take a nice example of something done by TEX and assume that what we do ourselves is equally okay. I still remember the tests we did with PDFTEX and hz. When Hàn Thế Thành and I discussed that with Hermann Zapf he was not surprised at all that no one saw a difference between the samples and instead was focusing on aspects that TEXies are told to look at, like two hyphens in a row.

A tool like TEX has a learning curve. If you don't like that just don't use it. If you think that someone doesn't like that, don't enforce this tool on that someone. And don't use (or lie with) statistics. Much better arguments are that it's a long-lived stable tool with a large user base and support. That it's not a waste of time. Watching a designer like Hermann Zapf draw shapes is more fun than watching click and point in heavily automated tools. It's probably also less fun to watch a TEXie converge towards a solution.

Spiders are resilient. Ants maybe even more. Ants will survive a nuclear blast (mutations might even bring them benefits), they can handle the impact of a meteorite, a change in climate won't harm them much. Their biggest enemy is probably us, when we try to wipe them out with poison. But, as long as they keep a low profile they're okay. TEX doesn't fit into the economic model as there is no turnaround involved, no paid development, it is often not seen at all, it's just a hit in a search engine and even then you might miss it (if only because no one pays for it being shown at the top).

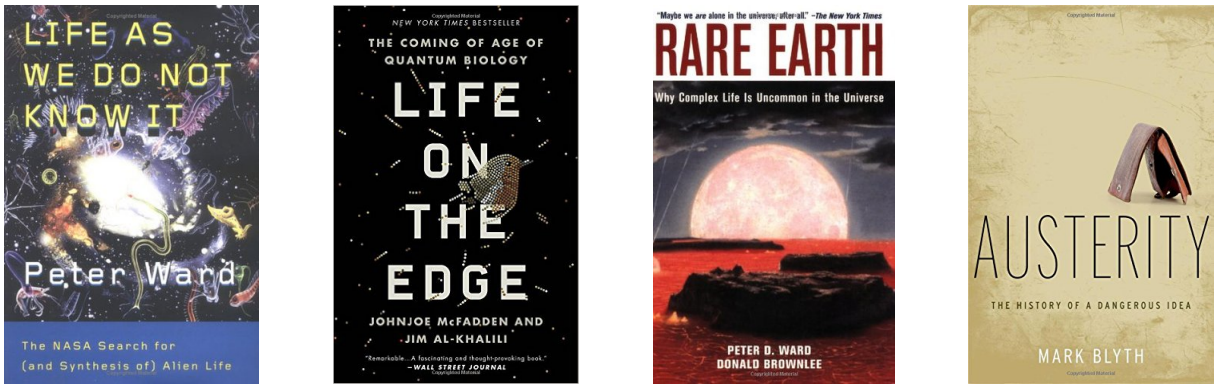
We can learn from that. Keeping a low profile doesn't trigger the competition to wipe you out. Many (open source) software projects fade away: some big company buys out the developer and stalls the project or wraps what they bought in their own stuff, other projects go professional and enterprise and alienate the original users. Yet others abort because the authors lose interest. Just like the ideals of socialism don't automatically mean that every attempt to implement it is a success, so not all open source and free software is good (natured) by principle either. The fact that communism failed doesn't mean that capitalism is better and a long term winner. The same applies to programs, whether successful or not.

Maybe we should be like the sheep. Dennett uses these animals as a clever species. They found a way to survive by letting themselves (unconsciously) be domesticated. The shepherd guarantees food, shelter and protection. He makes sure they don't get ill. Speaking biologically: they definitely made sure that many copies of their genes survived. Cows did the same and surprisingly many of them are related due to the fact that they share the same father (something now trying to be reverted). All TEX spinoffs relate to the same parent, and those that survived are those that were herded by user groups. We see bits and pieces of TEX end up in other applications. Hyphenation is one of them. Maybe we should settle for that small victory in a future hall of fame.

When I sit on my balcony and look at the fruit trees in my garden, some simple math can be applied. Say that one of the apple trees has 100 apples per year and say that this tree survives for 25 years (it's one of those small manipulated trees). That makes 2.500 apples. Without human intervention only a few of these apples make it into new trees, otherwise the whole world would be dominated by apple trees. Of course that tree now only survives because we permit it to survive, and for that it has to be humble (something that is very hard for modern Apples). Anyway, the apple tree doesn't look too unhappy.

A similar calculation can be done for birds that nest in the trees and under my roof. Given that the number of birds stays the same, most of energy spent on raising offspring is wasted. Nevertheless they seem to enjoy life. Maybe we should be content if we get one enthusiastic new user when we demonstrate TEX to thousands of potential users.

Maybe, coming back to the themes of the conference, we should not come up with these kinds of themes. We seem to be quite happy here. Talking about the things that we like, meeting people. We just have to make sure that we survive. Why not stay low under the radar? That way nothing will see us as a danger. Let's be like the ants and spiders, the invisible hive mind that carries our message, whatever that is.



astrobiology quantumbiology astrophysics economics

Figure 1.7

When Dennett discusses language he mentions (coined) words that survive in language. He also mentions that children pick up language no matter what. Their minds are made for it. Other animals don't do that: they listen but don't start talking back. Maybe TEX is just made for certain minds. Some like it and pick it up, while for others it's just noise. There's nothing wrong with that. Predilection can be a user property.

1.19 The unexpected

In a discussion with Dawkins the well-spoken astrophysicist Neil deGrasse Tyson brings up the following. We differ only a few percent in DNA from a chimp but quite a lot in brain power, so how would it be if an alien that differs a few percent (or more) passes by earth. Just like we don't talk to ants or chimps or whatever expecting an intelligent answer, whatever passes earth won't bother wasting time on us. Our rambling about the quality of typesetting probably sounds alien to many people who just want to read and who happily reflow a text on an ebook device, not bothered by a lack of quality.

We tend to take ourselves as reference. In "Rare Earth" Ward and Brownlee extrapolate the possibility of life elsewhere in the universe. They are not alone in thinking that while on one hand applying statistics to these formulas of possible life on planets there might also be a chance that we're the only intelligent species ever evolved. In a follow up, "Life as we do not know it" paleontologist and astrobiologist Ward (one of my favourite authors) discusses the possibility of life not based on carbon, which is not natural for a carbon based species. Carl Sagan once pointed out that an alien species looking down to earth can easily conclude that cars are the dominant species on earth and that the thingies crawling in and out them are some kind of parasites. So, when we look at the things that somehow end up on paper (as words, sentences, ornaments, etc.), what is dominant there? And is what we consider dominant really that dominant in the long run? You can look at a nice page as a whole and don't see the details of the content. Maybe beauty hides nonsense.

When TEXies look around they look to similar technologies. Commands in shells and solutions done by scripting and programming. This make sense in the perspective of survival. However, if you want to ponder alternatives, maybe not for usage but just for

fun, a completely different perspective might be needed. You must be willing to accept that communicating with a user of a WYSIWYG program might be impossible. If mutual puzzlement is a fact, then they can either be too smart and you can be too dumb or the reverse. Or both approaches can be just too alien, based on different technologies and assumptions. Just try to explain TEX to a kid 40 years younger or to an 80 year old grandparent for that matter. Today you can be very clever in one area and very stupid in another.

In another debate, Neil deGrasse Tyson asks Dawkins the question why in science fiction movies the aliens look so human and when they don't, why they look so strange, for instance like cumbersome sluggish snails. The response to that is one of puzzlement: the opponent has no reference of such movies. In discussions old TEXies like to suggest that we should convert young users. They often don't understand that kids live in a different universe.

How often does that happen to us? In a world of many billions TEX has its place and can happily coexist with other typesetting technologies. Users of other technologies can be unaware of us and even create wrong images. In fact, this also happens in the community itself: (false) assumptions turned into conclusions. Solutions that look alien, weird and wrong to users of the same community. Maybe something that I present as hip and modern and high-TEX and promising might be the opposite: backward, old-fashioned and of no use to others. Or maybe it is, but the audience is in a different mindset. Does it matter? Let's just celebrate that diversity. (So maybe, instead of discussing the conference theme, I should have talked about how I abuse LUATEX in controlling lights in my home as part of some IoT experiments.)

1.20 What drives us

I'm no fan of economics and big money talk makes me suspicious. I cannot imagine working in a large company where money is the drive. It also means that I have not much imagination in that area. We get those calls at the office from far away countries who are hired to convince us by phone of investments. Unfortunately mentioning that you're not at all interested in investments or that multiplying money is irrelevant to you does not silence the line. You have to actively kill such calls. This is also why I probably don't understand today's publishing world where money also dominates. Recently I ran into talks by Mark Blyth about the crisis (what crisis?) and I wish I could argue like he does when it comes to typesetting and workflows. He discusses quite well that most politicians have no clue what the crisis is about.

I think that the same applies to the management of publishers: many have no clue what typesetting is about. So they just throw lots of money into the wrong activities, just like the central banks seem to do. It doesn't matter if we TEXies demonstrate cheap and efficient solutions.

Of course there are exceptions. We're lucky to have some customers that do understand the issues at hand. Those are also the customers where authors may use the tools

themselves. Educating publishers, and explaining that authors can do a lot, might be a premise, predilection and prediction in one go! Forget about those who don't get it: they will lose eventually, unfortunately not before they have reaped and wasted the landscape.

Google, Facebook, Amazon, Microsoft and others invest a lot in artificial intelligence (or, having all that virtual cash, just buy other companies that do). They already have such entities in place to analyze whatever you do. It is predicted that at some point they know more about you then you know yourself. Reading Luke Dormehl's "The Formula" is revealing. So what will that do with our so-called (disputed by some) free will? Can we choose our own tools? What if a potential user is told that all his or her friends use WhateverOffice so they'd better do that too? Will subtle pressure lead them or even us users away from TEX? We already see arguments among TEXies, like "It doesn't look updated in 3 years, is it still good?" Why update something that is still valid? Will the community be forced to update everything, sort of fake updates. Who sets out the rules? Do I really need to update (or re-run) manuals every five years?

Occasionally I visit the Festo website. This is a (family owned) company that does research at the level that used to be common in large companies decades ago. If I had to choose a job, that would be the place to go to. Just google for "festo bionic learning network" and you understand why. We lack this kind of research in the field we talk about today: research not driven by commerce, short term profit, long term control, but because it is fundamental fun.

Last year Alan Braslau and I spent some time on BIBTEX. Apart from dealing with all the weird aspects of the APA standard, dealing with the inconsistently constructed author fields is a real pain. There have been numerous talks about that aspect here at BachoTEX by Jean-Michel Hufflen. We're trying to deal with a more than 30-year-old flawed architecture. Just look back over a curve that backtracks 30 years of exponential development in software and databases and you realize that it's a real waste of time and a lost battle. It's fine to have a text based database, and stable formats are great, but the lack of structure is appalling and hard to explain to young programmers. Compare that to the Festo projects and you realize that there can be more challenging projects. Of course, dealing with the old data can be a challenge, a necessity and eventually even be fun, but don't even think that it can be presented as something hip and modern. We should be willing to admit flaws. No wonder that Jean-Michel decided to switch to talking about music instead. Way more fun.

Our brains are massively parallel bio-machinery. Groups of neurons cooperate and compete for attention. Coming up with solutions that match what comes out of our minds demands a different approach. Here we still think in traditional programming solutions. Will new ideas about presenting information, the follow up on books come from this community? Are we the innovative Festo or are we an old dinosaur that just follows the fashion?

1.21 User experience

Here is a nice one. Harari spends many pages explaining that research shows that when an unpleasant experience has less unpleasantness at the end of the period involved, the overall experience is valued according to the last experience. Now, this is something we can apply to working with TEX: often, the more you reach the final state of typesetting the more it feels as all hurdles are in the beginning: initial coding, setting up a layout, figuring things out, etc.

It can only get worse if you have a few left-over typesetting disasters but there adapting the text can help out. Of course seeing it in a cheap bad print can make the whole experience bad again. It happens. There is a catch here: one can find lots of bad-looking documents typeset by TEX. Maybe there frustration (or indifference) prevails.

I sometimes get to see what kind of documents people make with CONTEXT and it's nice to see a good looking thesis with diverse topics: science, philosophy, music, etc. Here TEX is just instrumental, as what it is used for is way more interesting (and often also more complex) than the tool used to get it on paper. We have conferences but they're not about rocket science or particle accelerators. Proceedings of such conferences can still scream TEX, but it's the content that matters. Here somehow TEX still sells itself, being silently present in rendering and presentations. It's like a rootkit: not really appreciated and hard to get rid of. Does one discuss the future of rootkits other than in the perspective of extinction? So, even as an invisible rootkit, hidden in the workings of other programs, TEX's future is not safe. Sometimes, when you install a Linux system, you automatically get this large TEX installation, either because of dependencies or because it is seen as a similar toolkit as for instance Open (or is it Libre) Office. If you don't need it, that user might as well start seeing it as a (friendly) virus.

1.22 Conclusion

At some point those who introduced computers in typesetting had no problem throwing printing presses out of the window. So don't pity yourself if at some point in the near future you figure out that professional typesetting is no longer needed. Maybe once we let machines rule the world (even more) we will be left alone and can make beautiful documents (or whatever) just for the joy, not bothering if we use outdated tools. After all, we play modern music on old instruments (and the older rock musicians get, the more they seem to like acoustic).

There are now computer generated compositions that experienced listeners cannot distinguish from old school. We already had copies of paintings that could only be determined forgeries by looking at chemical properties. Both of these (artificial) arts can be admired and bring joy. So, the same applies to fully automated typeset novels (or runtime rendered ebooks). How bad is that really? You don't dig channels with your hand. You don't calculate logarithmic tables manually any longer.

However, one of the benefits of the Internet is watching and listening to great minds. Another is seeing musicians perform, which is way more fun that watching a computer (although googling for "animusic" brings nice visuals). Recently I ran into a wooden musical computer made by "Wintergatan" which reminded me of the "Paige Compositor" that we use in a LuaTeX cartoon. Watching something like that nicely compensates for a day of rather boring programming. Watching how the marble machine x (mmx) evolves is yet another nice distraction.

Now, the average age of the audience here is pretty high even if we consider that we get older. When I see solutions of ConTeXt users (or experts) posted by (young) users on the mailing list or stack exchange I often have to smile because my answer would have been worse. A programmable system invokes creative solutions. My criterion is always that it has to look nice in code and has some elegance. Many posted solutions fit. Do we really want more automation? It's more fun to admire the art of solutions and I'm amazed how well users use the possibilities (even ones that I already forgot).

One of my favourite artists on my weekly "check youtube" list is Jacob Collier. Right from when I ran into him I realized that a new era in music had begun. Just google for his name and "music theory interview" and you probably understand what I mean. When Dennett comments on the next generation (say up to 25) he wonders how they will evolve as they grow up in a completely different environment of connectivity. I can see that when I watch family members. Already long ago Greg Bear wrote the novel "Darwin's Children". It sets you thinking and when looking around you even wonder if there is a truth in it.

There are folks here at BachoTeX who make music. Now imagine that this is a conference about music and that the theme includes the word "future". Then, imagine watching that video. You see some young musicians, one of them probably one of the musical masterminds of this century, others instrumental to his success, for instance by wrapping up his work. While listening you realize that this next generation knows perfectly well what previous generations did and achieved and how they influenced the current. You see the future there. Just look at how old musicians reflect on such videos. (There are lots of examples of youth evolving into prominent musicians around and I love watching them). There is no need to discuss the future, in fact, we might make a fool of ourselves doing so. Now back to this conference. Do we really want to discuss the future? What we think is the future? Our future? Why not just hope that in the flow of getting words on a medium we play our humble role and hope we're not forgotten but remembered as inspiration.

One more word about predicting the future. When Arthur Clarke's "2001: A Space Odyssey" was turned into a movie in 1968, a lot of effort went into making sure that the not so far ahead future would look right. In 1996 scientists were asked to reflect on these predictions in "Hal's Legacy". It turned out that most predictions were plain wrong. For instance computers got way smaller (and even smaller in the next 20 years) while (self-aware) artificial intelligence had not arrived either. So, let's be careful in what we predict (and wish for).

1.23 No more themes

We're having fun here, that's why we come to BachoTEX (predilection). That should be our focus. Making sure that TEX's future is not so much in the cutting edge but in providing fun to its users (prediction). So we just have to make sure it stays around (premise). That's how it started out. Just watch at Don Knuth's 3:16 poster: via TEX and METAFONT he got in contact with designers and I wouldn't be surprised if that subproject was among the most satisfying parts. So, maybe instead of ambitious themes the only theme that matters is: show what you did and how you did it.

Advertising TEX 2

I can get upset when I hear TEXies boast about the virtues of TEX compared to for instance Microsoft Word. Not that I feel responsible for defending a program that I never use(d) but attacking something for no good reason makes not much sense to me. It is especially annoying when the attack is accompanied by a presentation that looks pretty bad in design and typography. The best advertisements for TEX should of course come from outside the TEX community, by people impressed by its capabilities. How many TEXies can really claim that Word is bad when they never tried to make something in it with a similar learning curve as they had in TEX or the same amount of energy spent in editing and perfecting a word-processor-made document.

In movies where computer technology plays a role one can encounter weird assumptions about what computers and programs can do. Run into a server room, pull one disk out of a RAID-5 array and get all information from it. Connect some magic device to a usb port of a phone and copy all data from it in seconds. Run a high speed picture or fingerprint scan on a computer (probably on a remote machine) and show all pictures flying by. Okay, it's not so far from other unrealistic aspects in movies, like talking animals, so maybe it is just a metaphor for complexity and speed. When zapping channels on my television I saw figure 2.1 and as the media box permits replay I could make a picture. I have no clue what the movie was about or what movie it was so a reference is lacking here. Anyway it's interesting that seeing a lot of TEX code flying by can impress someone: the viewer, even if no TEXie will ever see that on the console unless in some error or tracing message and even then it's hard to get that amount. So, the viewer will never realize that what is seen is definitely not what a TEXie wants to see.

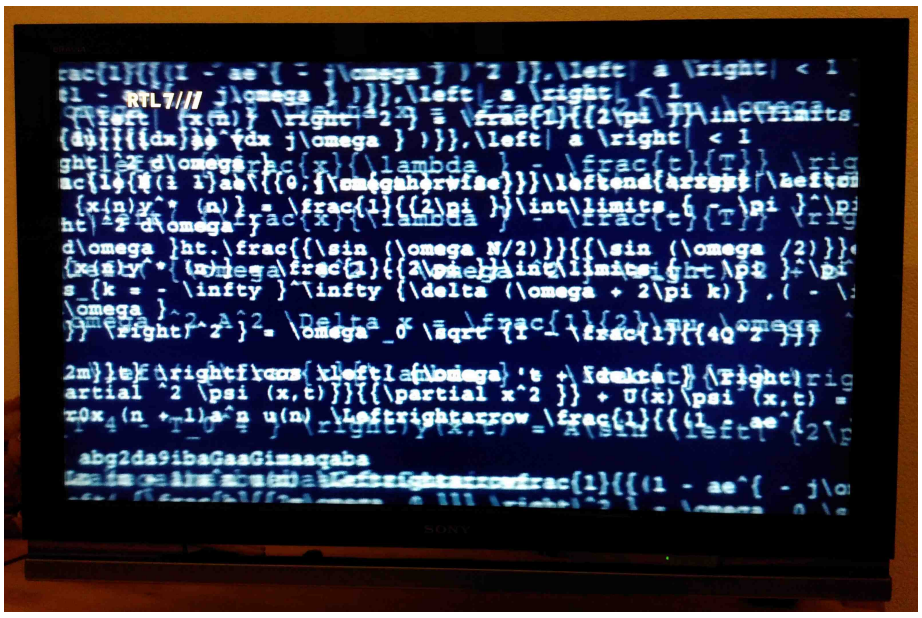


Figure 2.1 TEX in a movie

So, as that kind of free advertisement doesn't promote TEX well, what of an occasional mentioning of TEX in highly-regarded literature? When reading "From bacteria to Bach and back, the evolution of minds" by Daniel Dennett I ran into the following:

> "In Microsoft Word, for instance, there are the typographical operations of superscript and subscript, as illustrated by
>
> $\text{base}^{\text{power}}$
>
> and
>
> $\text{human}_{\text{female}}$
>
> But try to add another superscript to $\text{base}^{\text{power}}$—it *should* work, but it doesn't! In mathematics, you can raise powers to powers to powers forever, but you can't get Microsoft Word to display these (there are other text-editing systems, such as TeX, that can). Now, are we sure that human languages make use of true recursion, or might some or all of them be more like Microsoft Word? Might our interpretation of grammars as recursive be rather an elegant mathematical idealization of the actual "moving parts" of a grammar?"

Now, that book is a wonderfully interesting read and the author often refers to other sources. When one reads some reference (with a quote) then one assumes that what one reads is correct, and I have no reason to doubt Dennett in this. But this remark about TEX has some curious inaccuracies.[1]

First of all a textual raise or lower is normally not meant to be recursive. Nesting would have interesting consequences for the interline space so one will avoid it whenever possible. There are fonts that have superscript and subscript glyphs and even UNICODE has slots for a bunch of characters. I'm not sure what Word does: take the special glyph or use a scaled down copy?

Then there is the reference to TEX where we can accept that the "E" is not lowered but just kept as a regular "e". Actually the mentioning of nested scripts refers to typesetting math and that's what the superscripts and subscripts are for in TEX. In math mode however, one will normally raise or lower symbols and numbers, not words: that happens in text mode.

While Word will use the regular text font when scripting in text mode, a TEX user will either have to use a macro to make sure that the right size (and font) is used, or one can revert to math mode. But how to explain that one has to enter math and then explicitly choose the right font? Think of this:

```
efficient\high{efficient} or
```

[1] Of course one can wonder in general that when one encounters such an inaccuracy, how valid other examples and conclusions are. However, consistency in arguments and confirmation by other sources can help to counter this.

```
efficient$^{\text{efficient}}$ or \par
{\bf efficient\high{efficient} or
efficient$^{\text{efficient}}$}
```

Which gives (in Cambria)

efficient$^{\text{efficient}}$ or efficient$^{\text{efficient}}$ or

efficient$^{\text{efficient}}$ or efficient$^{\text{efficient}}$

Now this,

```
efficient\high{efficient\high{efficient}} or
efficient$^{\text{efficient$^{\text{efficient}}$}}$ or \par
{\bf efficient\high{efficient\high{efficient}} or
efficient$^{\text{efficient$^{\text{efficient}}$}}$}
```

will work okay but the math variant is probably quite frightening at a glance for an average Word user (or beginner in TEX) and I can understand why someone would rather stick to click and point.

efficient$^{\text{efficient}^{\text{efficient}}}$ or efficient$^{\text{efficient}^{\text{efficient}}}$ or

efficient$^{\text{efficient}^{\text{efficient}}}$ or efficient$^{\text{efficient}^{\text{efficient}}}$

Oh, and it's tempting to try the following:

```
efficient{\addff{f:superiors}efficient}
```

but that only works with fonts that have such a feature, like Cambria:

efficient$^{\text{efficient}}$

To come back to Dennett's remark: when typesetting math in Word, one just has to switch to the math editing mode and one can have nested scripts! And, when using TEX one should not use math mode for text scripts. So in the end in both systems one has to know what one is doing, and both systems are equally capable.

The recursion example is needed in order to explain how (following recent ideas from Chomsky) for modern humans some recursive mechanism is needed in our wetware. Now, I won't go into details about that (as I can only mess up an excellent explanation) but if you want to refer to TEX in some way, then expansion[2] of (either combined or not) snippets of knowledge might be a more interesting model than recursion, because much of what TEX is capable of relates to expansion. But I leave that to others to explore.[3]

[2] Expanding macros actually works well with tail recursion.

[3] One quickly starts thinking of how expandafter, `noexpand`, `unexpanded`, `protected` and other primitives can be applied to language, understanding and also misunderstanding.

Now, comparing TEX to Word is always kind of tricky: Word is a text editor with typesetting capabilities and TEX is a typesetting engine with programming capabilities. Recursion is not really that relevant in this perspective. Endless recursion in scripts makes little sense and even TEX has its limits there: the TEX math engine only distinguishes three levels (text, script and scriptscript) and sometimes I'd like to have a level more. Deeper nesting is just more of scriptscript unless one explicitly enforces some style. So, it's recursive in the sense that there can be many levels, but it also sort of freezes at level three.

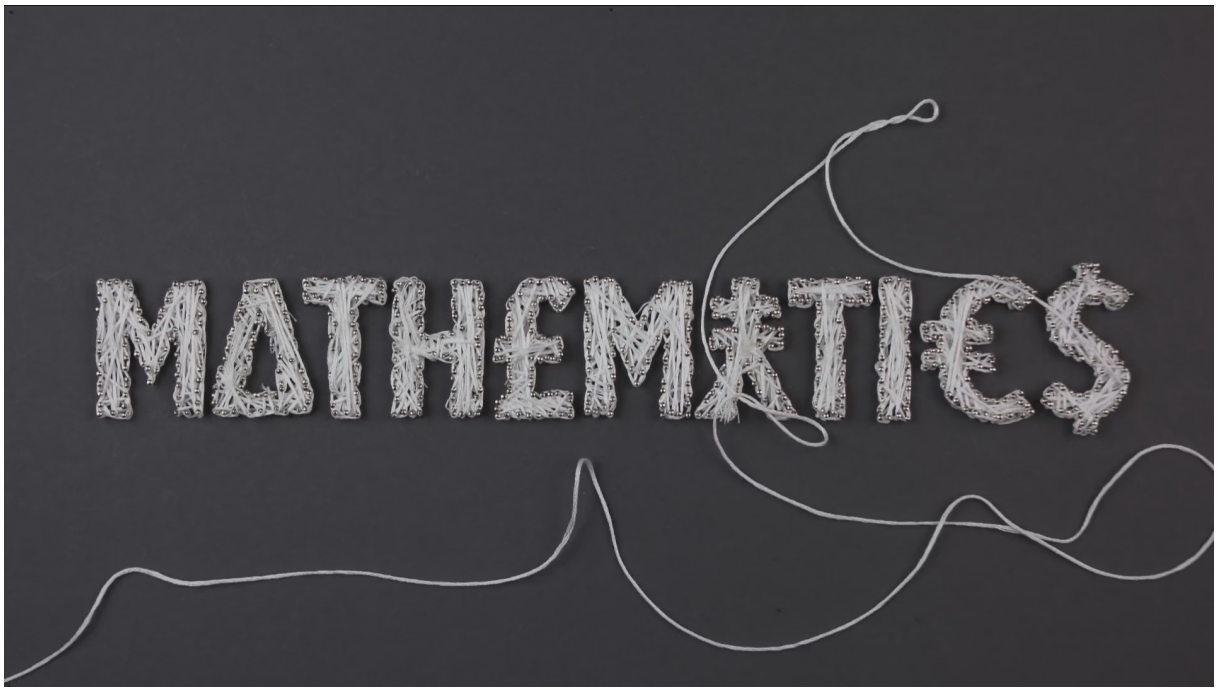


Figure 2.2 Nicer than TEX

I love TEX and I like what you can do with it and it keeps surprising me. And although mathematics is part of that, I seldom have to typeset math myself. So, I can't help that figure 2.2 impresses me more. It even has the so-familiar-to-TEXies dollar symbols in it: the poem "Poetry versus Orchestra" written by Hollie McNish, music composed by Jules Buckley and artwork by Martin Pyper (I have the DVD but you can also find it on YOUTUBE). It reminds me of Don Knuth's talk at a TUG meeting. In TUGBOAT 31:2 (2010) you can read Don's announcement of his new typesetting engine iTEX: "Output can be automatically formatted for lasercutters, embroidery machines, 3D printers, milling machines, and other CNC devices . . .". Now that is something that Word can't do!

Why use TEX? 3

3.1 Introduction

Let's assume that you know what TEX is: a program that interprets a language with the same name that makes it possible to convert (tagged) input into for instance PDF. For many of its users it is a black box: you key in some text, hit a button and get some typeset result in return. After a while you start tweaking this black box, meet other users (on the web), become more fluent and stick to it forever.

But now let's assume that you don't know TEX and are in search of a system that helps you create beautiful documents in an efficient way. When your documents have a complex structure you are probably willing to spend some time on figuring out what the best tool is. Even if a search lets you end up with something called TEX, a three letter word with a dropped E, you still don't know what it is. Advertisement for TEX is often pretty weak. It's rather easy to point to the numerous documents that can be found on the web. But what exactly does TEX do and what are its benefits? In order to answer this we need to know who you are: an author, editor, an organization that deals with documents or needs to generate readable output, like publishers do.

3.2 Authors

We start with authors. Students of sciences that use mathematics don't have much of a choice. But most of these documents hardly communicate the message that "Everyone should use TEX." or that "All documents produced by TEX look great." but they do advocate that for rendering math it is a pretty good system. The source code of these documents often look rather messy and unattractive and for a non-math user it can be intimidating. Choosing some lightweight click-and-ping alternative looks attractive. lightweight click-and-ping alternative looks attractive.

Making TEX popular is not going to happen by convincing those who have to write an occasional letter or report. They should just use whatever suits them. On the other hand if you love consistency, long term support, need math, are dealing with a rare language or script, like to reuse content, prefer different styling from one source, use one source for multiple documents, or maybe love open source tools, then you are a candidate. Of course there is a learning curve but normally you can master TEX rather fast and once you get the hang of it there's often no way back. But you always need to invest a bit beforehand.

So what authors are candidates for TEX? It could be that TEX is the only tool that does the job. If so, you probably learned that from someone who saw you struggle or had the same experience and wrote or talked about it somewhere. In that case using TEX for creating just one document (like a thesis) makes sense. Otherwise, you should really

wonder if you want to invest time in a tool that you probably have to ditch later on as most organizations stick to standard (commercial) word processing tools.

Talking to customers we are often surprised that people have heard about TEX, or even used it for a few documents in college. Some universities just prescribe the use of TEX for reporting, so not much of a choice there. Memories are normally rather positive in the sense that they know that it can do the job and that it's flexible.

User group journals, presentations at TEX meetings, journals, books and manuals that come with TEX macro packages can all be used to determine if this tool suits an author. Actually, I started using TEX because the original TEXbook had some magic, and reading it was just that: reading it, as I had no running implementation. A few years later, when I had to write (evolving) reports, I picked up again. But I'm not a typical user.

3.3 Programmers

When you are a programmer who has to generate reports, for instance in PDF, or write manuals, then TEX can really be beneficial. Of course TEX is not always an obvious choice, but if you're a bit able to use it it's hard to beat in quality, flexibility and efficiency. I'm often surprised that companies are willing to pay a fortune for functionality that basically comes for free. Programmers are accustomed to running commands and working in a code editor with syntax highlighting so that helps too. They too recognize when something can be done more efficiently.

When you need to go from some kind of input (document source, database, generated) to some rendered output there currently are a few endpoints: a (dynamic) HTML page, a PDF document, something useable in a word processor, or a representation using the desktop user interface. It's the second category where TEX is hard to beat but even using TEX and METAPOST for creating a chart can make sense.

There are of course special cases where TEX fits in nicely. Say that you have to combine PDF documents. There are numerous tools to do that and TEX is one. The advantage of TEX over other tools is that it's trivial to add additional text, number pages, provide headers and footers. And it will work forever. Why? Because TEX has been around for decades and will be around for decades to come. It's an independent component. The problem with choosing for TEX is that the starting point is important. The question is not "What tool should I use?" but "What problem do I need to solve?". An open discussion about the objectives and possibilities is needed, not some checklist based on assumptions. If you don't know TEX and have never worked with a programmable typesetting environment, you probably don't see the possibilities. In fact, you might even choose for TEX for the wrong reasons.

The problem with this category of users is that they seldom have the freedom to choose their tools. There are not that many jobs where the management is able to recognize the clever programmer who can determine that TEX is suitable for a lot of jobs and can save money and time. Even the long term availability and support is not an argument since

not only most tools (or even apis) changes every few years but also organizations themselves change ownership, objectives, and personnel on a whim. The concept of 'long term' is hard to grasp for most people (just look at politics) and it's only in retrospect that one can say 'We used that toolkit for over a decade.'

3.4 Organizations

Authors (often) have the advantage that they can choose themselves: they can use what they like. In practice any decent programmer is able to find the suitable tools but convincing the management to use one of them can be a challenge. Here we're also talking of 'comfort zones': you have to like a tool(chain). Organizations normally don't look for TEX. Special departments are responsible for choosing and negotiating whatever is used in a company. Unfortunately companies don't always start from the open question "We have this problem, we want to go there, what should we do?" and then discuss options with for instance those who know TEX. Instead requirements are formulated and matches are found. The question then is "Are these requirements cut in stone?" and if not (read: we just omit some requirements when most alternatives don't meet them), were other requirements forgotten? Therefore organizations can end up with the wrong choice (using TEX in a situation where it makes no sense) or don't see opportunities (not using TEX while it makes most sense). It doesn't help that a hybrid solution (use a mix of TEX and other tools) is often not an option. Where an author can just stop using a tool after a few days of disappointment, and where a programmer can play around a bit before making a choice, an organization probably best can start small with a proof of concept.

Let's take a use case. A publisher wants to automatically convert XML files into PDF. One product can come from multiple sources (we have cases where thousands of small XML files combine into one final product). Say that we have three different layouts: a theory book, a teachers manual and an answer book. In addition special proofing documents have to be rendered. The products might be produced on demand with different topics in any combination. There is at least one image and table per page, but there can be more. There are color and backgrounds used, tables of contents generated, there is extensive cross referencing and an index. Of course there is math.

Now let's assume an initial setup costs 20K Euro and, what happens often when the real products show up, a revision after one year takes the same amount. We also assume 10K for the following eight years for support. So, we end up with 120K over 10 years. If one goes cheap we can consider half of that, or we can be pessimistic and double the amount.

The first year 10K pages are produced, the second year 20K and after that 30K per year. So, we're talking of 270K pages. If we include customer specific documents and proofing we might as well end up with a multiple of that.

So, we have 120K Euro divided by 270K pages or about half an Euro per page. But likely we have more pages so it costs less. If we double the costs then we can assume

that some major changes took place which means more pages. In fact we had projects where the layout changed, all documents were regenerated and the costs were included in the revision, so far from double. We also see many more pages being generated so in practice the price per page drops below half an Euro. The more we process the cheaper it gets and one server can produce a lot of pages!

Now, the interesting bit of such a calculation is that the costs only concern the hours spent on a solution. A TEX based system comes for free and there are no license costs. Whatever alternative is taken, even if it is as flexible, it will involve additional costs. From the perspective of costs it's very hard to beat TEX. Add to that the possibility for custom extensions, long term usage and the fact that one can adapt the system. The main question of course is: does it do the job. The only way to find out is to either experiment (which is free), consult an expert (not free, but then needed anyway for any solution) or ask an expert to make a proof of concept (also not free but relatively cheap and definitely cheaper than a failure). In fact, before making decisions about what solution is best it might be a good idea to check with an expert anyway, because more or less than one thinks might be possible. Also, take into account that the TEX ecosystem is often one of the first to support new technologies, and normally does that within its existing interface. And there is plenty of free support and knowledge available once you know how to find it. Instead of wasting time and money on advertisement and fancy websites, effort goes into support and development. Even if you doubt that the current provider is around in the decade to come, you can be sure that there will be others, simply because TEX attracts people. Okay, it doesn't help that large companies like to out source to far–far–away and expect support around the corner, so in the end they might kill their support chain.

When talking of TEX used in organizations we tend to think of publishers. But this is only a small subset of organizations where information gets transformed into something presentable. For small organizations the choice for TEX can be easy: costs, long term stability, knowing some experts are driving forces. For large organizations these factors seem (at least to us) hardly relevant. We've (had) projects where actually the choice for using a TEX based solution was (in retrospect) a negative one: there was no other tool than this relatively unknown thing called TEX. Or, because the normal tools could not be used, one ended up with a solution where (behind the scenes) TEX is used, without the organization knowing it. Or, it happened that the problem at hand was mostly one that demands in-depth knowledge of manipulating content, cleaning up messy data, combining resources (images or PDF documents), all things that happen to be available in the perspective of TEX. If you can solve a hard to solve problem for them then an organization doesn't care what tool you use. What does matter is that the solution runs forever, that costs are controllable and above all, that it "Just works." And if you can make it work fast, that helps too. We can safely claim that when TEX is evaluated as being a good option, that in the end it always works out quite well.

Among arguments that (large) organizations like to use against a choice for TEX (or something comparable) are the size of the company that they buy their solution from, the expected availability for support, and the wide-spread usage of the tool at hand.

One can wonder if it also matters that many vendors change ownership, change products every few years, change license conditions when they like, charge a lot for support or just abort a tool chain. Unfortunately when that happens those responsible for choosing such a system can have moved on to another job, so this is seldom part of an evaluation. For the supplier the other side of the table is just as much of a gamble. In that respect, an organization that wants to use an open source (and/or free) solution should realize that getting a return on investment on such a development is pretty hard to achieve. So, who really takes the risk for writing open source?

For us, the reason to develop ConTeXt and make it open is that it fits in our philosophy and we like the community. It is actually not really giving us an advantage commercially: it costs way more to develop, support and keep up-to-date than it will ever return. We can come up with better, faster and easier solutions and in the end we pay the price because it takes less time to cook up styles. So there is some backslash involved because commercially a difficult solution leads to more billable hours. Luckily we tend to avoid wasting time so we improve when possible and then it ends up in the distributed code. And, once the solution is there, anyone can use it. Basically also for us it's just a tool, like the operating system, editor and viewer are. So, what keep development going is mostly the interaction with the community. This also means that a customer can't really demand functionality for free: either wait for it to show up or pay for it (which seldom happens). Open source is not equivalent with "You get immediately what you want because someone out there writes the code.". There has to be a valid reason and often it's just users and meetings or just some challenge that drives it.

This being said, it is hard to convince a company to use TeX. It has to come from users in the organization. Or, what we sometimes see with publishers, it comes with an author team or acquired product line where it's the only option. Even then we seldom see transfer to other branches in the organizations. No one seems to wonder "How on earth can that XML to PDF project produce whatever output in large quantities in a short period of time" while other (past) projects failed. It probably relates to the abstraction of the process. Even among TeX users it can be that you demonstrate something with a click on a button and that many years afterwards someone present at that moment tells you that they just discovered that this or that can be done by hitting a button. I'm not claiming that TeX is the magic wand for everything but in some areas it's pretty much ahead of the pack. Go to a TeX user meeting and you will be surprised about the accumulated diverse knowledge present in the room. It's user demand that drives ConTeXt development, not commerce.

3.5 Choosing

So, where can one find information about TeX and friends? On the web one has to use the right search keys, so adding `tex` helps: `context tex` or `xml tex pdf` and so on. Can one make a fancy hip website, sure, but it being a life-long, already old and mature environment, and given that it comes for free, or is used low-budget, not much effort and money can be spent on advertising it. A benefit is that no false promises and hypes are made either. If you want to know more, just ask the right folks.

For all kind of topics one can find interesting videos and blogs. One can subscribe to channels on YouTube or join forums. Unfortunately not that many bloggers or vloggers or podcasters come up with original material every time, and often one starts to recognize patterns and will get boring by repetition of wisdom and arguments. The same is true for manuals. Is a ten year old manual really obsolete? Should we just recompile it to fake an update while in fact there has been no need for it? Should we post twenty similar presentations while one can do? (If one already wants to present the same topic twenty times in the first place?) Maybe one should compare TEX with cars: they became better over time and can last for decades. And no new user manual is needed.

As with blogs and vlogs advertising TEX carries the danger for triggering political discussions and drawing people into discussions that are not pleasant: TEX versus some word processor, open versus closed source, free versus paid software, this versus that operating system, editor such or editor so.

To summarize, it's not that trivial to come up with interesting information about TEX, unless one goes into details that are beyond the average user. And those who are involved are often involved for a long time so it gets more complex over time. User group journals that started with tutorials later on became expert platforms. This is a side effect of being an old and long-term toolkit. If you run into it, and wonder if it can serve your purpose, just ask an expert.

Most TEX solutions are open source and come for free as well. Of course if you want a specific solution or want support beyond what is offered on mailing lists and forums you should be willing to pay for the hours spent. For a professional publisher (of whatever kind) this is not a problem, if only because any other solution also will cost something. It is hard to come up with a general estimate. A popular measure of typesetting costs is the price per page, which can range from a couple of euro's per page to two digit numbers. We've heard of cases where initial setup costs were charged. If not much manual intervention is needed a TEX solution mostly concerns initial costs.

Let's return to the main question "Why use TEX?" in which you can replace TEX by one of the macro packages build on top of it, for instance CONTEXT. If an (somewhat older) organization considers using TEX it should also ask itself, why it wasn't considered long ago already? For sure there have been developments in TEX engines (in CONTEXT we use LUATEX) as well as possibilities of macro packages but if you look at the documents produced with them, there is not that much difference with decades ago. Processing has become faster, some things have become easier, but new technologies have always been supported as soon at they showed up. Advertising is often just repeating an old message.

The TEX ecosystem was among the first in supporting for instance OPENTYPE, and the community even made sure that there were free fonts available. A format like PDF was supported as soon as it shows up and TEX was the first to demonstrate what advanced features were there and how way it was to adapt to changes. Processing XML using TEX has never been a big deal and if that is a reason to look at this already old and mature technology, then an organization can wonder if years and opportunities (for instance for

publishing on demand or easy updating of manuals) have been lost. Of course there are (and have been) alternative tools but the arguments for using TEX or not are not much different now. It can be bad marketing of open and free software. It can be that TEX has been around too long. It can also be that its message was not understood yet. On the other hand, in software development it's quite common to reinvent wheels and present old as new. It's never to late to catch on.

What's to stay, what's to go 4

4.1 Introduction

The following text was written as preparation for a 2018 talk at BachoTEX, which has this theme. It's mostly a collection of thoughts. It was also more meant as a wrapup for the presentation (possibly with some discussions) than an article.

4.2 Attraction

There are those movies where some whiz-kid sits down behind a computer, keys in a few commands, and miracles happen. Ten fingers are used to generate programs that work immediately. It's no problem to bypass firewalls. There is no lag over network connections. Checking massive databases is no big deal and there's even processing power left for real time visualization or long logs to the terminal.

How boring and old fashioned must a regular edit–run–preview cycle look compared to this. If we take this 2018 movie reality as reference, in a time when one can suck a phone empty with a simple connection, pull a hard drive from a raid five array and still get all data immediately available, when we can follow realtime whoever we want using cameras spread over the country, it's pretty clear that this relatively slow page production engine TEX has no chance to survive, unless we want to impress computer illiterate friends with a log flying by on the console (which in fact is used in movies to impress as well).

On YouTube you can find these (a few hours) sessions where Jacob Collier harmonizes live in one of these Digital Audio Workstation programs. A while later on another channel June Lee will transcribe these masterpieces into complex sheets of music by ear. Or you can watch the weekly Wintergatan episodes on building the Marble Machine from wood using drilling, milling, drawing programs etc. There are impressive videos of multi-dimensional led arrays made by hand and controlled by small computers and robots that solve Rubic Cubes. You can be impressed by these Animusic videos, musicians show their craftmanship and interesting informative movies are all over the place. I simply cannot imagine millions of kids watching a TEX style being written in a few hours. It's a real challenge for an attention span. I hope to be proven wrong but I fear that for the upcoming generation it's probably already too late because the 'whow' factor of TEX is low at first encounter. Although: picking up one of Don Knuths books can have that effect: a nice mixture of code, typesetting and subtle graphics, combined with great care, only possible with a system like TEX.

: Biology teaches us that 'cool' is not a recipe for 'survival'. Not all designs by nature look cool, and it's only efficiency and functionality that matters. Beauty sometimes matters too but many functional mechanisms can do without. So far

TEX and its friends were quite capable to survive so there must be something in it that prevents it to be discarded. But survival is hard to explain. So far TEX just stayed around but lack of visual attraction is a missing competitive trait.

4.3 Satisfaction

Biology also teaches us that chemistry can overload reason. When we go for short-term pleasure instead of long-term satisfaction (Google for Simon Sinek on this topic), addiction kicks in (for instance driven by crossing the dopamine thresholds too often, Google for Robert Sapolsky). Cool might relate more to pleasure while satisfaction relates to an effort. Using TEX is not that cool and often takes an effort. But the results can be very satisfying. Where 'cool' is rewarding in the short term, 'satisfaction' is more a long term effect. So, you probably get the best (experience) out of TEX by using it a lifetime. That's why we see so many old TEXies here: many like the rewards.

If we want to draw new users we run into the problem that humans are not that good in long term visions. This means that we cannot rely on showing cool (and easy) features but must make sure that the long term reward is clear. We can try to be 'cool' to draw in new users, but it will not be the reason they stay. Instant success is important for kids who have to make a report for school, and a few days "getting acquainted with a program" doesn't fit in. It's hard to make kids addicted to TEX (which could be a dubious objective).

: As long as the narrative of satisfaction can be told we will see new users. Meetings like BachoTEX is where the narrative gets told. What will happen when we no longer meet?

4.4 Survival

Survival relates to improvements, stability and discarding of weak aspects. Unfortunately that does not work out well in practice. Fully automated multi–columns typesetting with all other elements done well too (we just mention images) is hard and close to impossible for arbitrary cases, so nature would have gotten rid of it. Ligatures can be a pain especially when the language is not tagged and some kind of intelligence is needed to selectively disable them. They are the tail of the peacock: not that handy but meant to be impressive. Somehow it stayed around in automated typesetting, in biology it would be called a freak of nature: probably a goodbye in wildlife. And how about page breaks on an electronic device: getting rid of them would make the floating figures go away and remove boundary conditions often imposed. It would also make widows and clubs less of a problem. One can even wonder if with page breaks the windows and clubs are the biggest problems, and if one can simply live with them. After all, we can live with our own bodily limitations too. After all, (depending on what country you live in) you can also live with bad roads, bad weather, polution, taxes, lack of healthcare for many, too much sugar in food, and more.

: Animals or plants that can adapt to live on a specific island might not survive elsewhere. Animals or plants introduced in an isolated environment might quickly dominate and wipe out the locals. What are the equivalents in our TEX ecosystem?

4.5 Niches

But arguments will not help us determine if TEX is the fittest for survival. It's not a rational thing. Humans are bad in applying statistics in their live, and looking far ahead is not a treat needed to survive. Often nature acts in retrospect. (Climbing mount probability by Richard Dawkins). So, it doesn't matter if we save time in the future if it complicates the current job. If governments and companies cannot look ahead and act accordingly, how can we extrapolate software (usage) or more specifically typesetting demands. Just look at the political developments in the country that hosts this conference. Could we have predicted the diminishing popularity of the EU (and disturbing retrograde political mess in some countries) of 2018 when we celebrated the moment Poland joining the EU at a BachoTEX campfire?

Extrapolating the future quality of versions of TEX or macro packages also doesn't matter much. With machine learning and artificial intelligence around the corner and with unavoidable new interfaces that hook into our brains, who knows what systems we need in the future. A generic flexible typesetting system is probably not the most important tool then. When we discuss quality and design it gets personal so a learning system that renders neutrally coded content into a form that suits an individual, demands a different kind of tool than we have now.

On the short term (our live span) it makes more sense to look around and see how other software (ecosystems) fare. Maybe we can predict TEX's future from that. Maybe we can learn from others mistakes. In the meantime we should not flatter ourselves with the idea that a near perfect typesetting system will draw attention and be used by a large audience. Factors external to the community play a too important role in this.

: It all depends on how well it fits into a niche. Sometimes survival is only possible by staying low on the radar. But just as we destroy nature and kill animals competing for space, programs get driven out of the software world. On a positive note: in a project that provides open (free) math for schools students expressed to favour a printed book over WEB-only (one curious argument for WEB was that it permits easier listening to music at the same time).

4.6 Dominance

Last year I installed a bit clever (evohome) heating control system. It's probably the only "working out of the box" system that supports 12 zones but at the same time it has a rather closed interface as any other. One can tweak a bit via a web interface but that one works by a proxy outside so there is a lock in. Such a system is a gamble because

it's closed and we're talking of a 20 year investment. I was able to add a layer of control (abusing LuaTeX as Lua engine and ConTeXt as library) so let's see. When I updated the boiler I also reconfigured some components (like valves) and was surprised how limited upgrading was supported. One ends up with lost settings and weird interference and it's because I know a bit of programming that I kept going and managed to add more control. Of course, after a few weeks I had to check a few things in the manuals, like how to enter the right menu.

So, as the original manuals are stored somewhere, one picks up the smart phone and looks for the manual on the web. I have no problem with proper PDF as a manual but why not provide a simple standard format document alongside the fancy folded A3 one. Is it because it's hard to produce different instances from one source? Is it because it takes effort? We're talking of a product that doesn't change for years.

: The availability of flexible tools for producing manuals doesn't mean that they are used as such. They don't support the survival of tools. Bad examples are a threat. Dominant species win.

4.7 Extinction

When I was writing this I happened to visit a bookshop where I always check the SciFi section for new publications. I picked out a pocket and wondered if I had the wrong glasses on. The text was wobbling and looked kind of weird. On close inspection indeed the characters were kind of randomly dancing on the baseline and looked like some 150 DPI (at most) scan. (By the way, I checked this the next time I was there by showing the book to a nephew.) I get the idea that quite some books get published first in the (more expensive) larger formats, so normally I wait till a pocket size shows up (which can take a year) so maybe here I had to do with a scan of a larger print scaled down.

What does that tell us? First of all that the publisher doesn't care about the reader: this book is just unreadable. Second, it demonstrates that the printer didn't ask for the original PDF file and then scaled down the outline copy. It really doesn't matter in this case if you use some high quality typesetting program then. It's also a waste of time to talk to such publishers about quality typesetting. The printer probably didn't bother to ask for a PDF file that could be scaled down.

: In the end most of the publishing industry will die and this is just one of the symptoms. Typesetting as we know it might fade away.

4.8 Desinterest

The newspaper that I read has a good reputation for design. But why do they need to drastically change the layout and font setup every few years? Maybe like an animal marking his or her territory a new department head also has to put a mark on the layout. Who knows. For me the paper became pretty hard to read: a too light font that suits

none of the several glasses that I have. So yes, I spend less time reading the paper. In a recent commentary about the 75 year history of the paper there was a remark about the introduction of a modern look a few decades ago by using a sans serif font. I'm not sure why sans is considered modern (most handwriting is sans) and to me some of these sans fonts look pretty old fashioned compared to a modern elegant serif (or mix).

: If marketing and fashion of the day dominate then a wrong decision can result in dying pretty fast.

4.9 Persistence

Around the turn of the century I had to replace my CD player and realized that it made more sense to invest in ripping the CD's to FLAC files and use a decent DAC to render the sound. This is a generic approach similar to processing documents with TEX and it looks as future proof as well. So, I installed a virtual machine running SlimServer and bought a few SlimDevices, although by that time they were already called SqueezeBoxes.

What started as an independent supplier of hardware and an open source program had gone the (nowadays rather predictable) route of a buy out by a larger company (Logitech). That company later ditched the system, even if it had a decent share of users. This "start something interesting and rely on dedicated users", then "sell yourself (to the highest bidder)" and a bit later "accept that the product gets abandoned" is where open source can fail in many aspects: loyal users are ignored and offended with the original author basically not caring about it. The only good thing is that because the software is open source there can be a follow up, but of course that requires that there are users able to program.

I have 5 small boxes and a larger transporter so my setup is for now safe from extinction. And I can run the server on any (old) LINUX or MS WINDOWS distribution. For the record, when I recently connected the 20 year old Cambridge CD2 I was surprised how well it sounded on my current headphones. The only drawback was that it needs 10 minutes for the transport to warm up and get working.

In a similar fashion I can still use TEX, even when we originally started using it with the only viable quality DVI to POSTSCRIPT backend at that time (DVIPSONE). But I'm not so sure what I'd done if I had not been involved in the development of PDFTEX and later LUATEX. As an average user I might just have dropped out. As with the CD player, maybe someone will dust off an old TEX some day and maybe the only hurdle is to get it running on a virtual retro machine. Although ... recently I ran into an issue with a virtual machine that didn't provide a console after a KVM host update, so I'm also getting pessimistic about that escape for older programs. (Not seldom when a library update is forced into the LUATEX repository we face some issue and it's not something the average user want (or is able to) cope with.)

: Sometimes it's hard to go extinct, even when commerce interfered at some point. But it does happen that users successfully take (back) control.

4.10 Freedom

If you buy a book originating in academia written and typeset by the author, there is a chance that it is produced by some flavour of TEX and looks quite okay. This is because the author could iterate to the product she or he likes. Unfortunately the web is also a source of bad looking documents produced by TEX. Even worse is that many authors don't even bother to set up a document layout properly, think about structure and choose a font setup that matches well. One can argue that only content matters. Fine, but than also one shouldn't claim quality simply because TEX has been used.

I've seen examples of material meant for bachelor students that made me pretend that I am not familiar with TEX and cannot be held responsible. Letter based layouts on A4 paper, or worse, meant for display (or e-book devices) without bothering to remove the excessive margins. Then these students are forced to use some collaborative TEX environment, which makes them dependent on the quality standards of fellow students. No wonder that one then sees dozens of packages being loaded, abundant copy and paste and replace of already entered formulas and interesting mixtures of inline and display math, skips, kerns and whatever can help to make the result look horrible.

: Don't expect enthusiast new users when you impose TEX but take away freedom and force folks to cooperate with those with lesser standards. It will not help quality TEX to stay around. You cannot enforce survival, it just happens or not, probably better with no competition or with a competition so powerful that it doesn't bother with the niches. In fact, keeping a low profile might be best! The number of users is no indication of quality, although one can abuse that statistic selectively?

4.11 Diversity

Diversity in nature is enormous. There are or course niches, but in general there are multiple variants of the same. When humans started breeding stock or companion animals diversity also was a property. No one is forcing the same dog upon everyone or the same cow. However, when industrialization kicks in things become worse. Many cows in our country share the same dad. And when we look at for instance corn, tomatoes or whatever dominance is not dictated by what nature figures out best, but by what commercially makes most sense, even if that means that something can't reproduce by itself any longer.

In a similar way the diversity of methods and devices to communicate (on paper) at some point turns into commercial uniformity. The diversity is simply very small, also in typesetting. And even worse, a user even has to defend her/himself for a choice of system (even in the TEX community). It's just against nature.

: Normally something stays around till it no longer can survive. However, we humans have a tendency to destroy and commerce is helping a hand here. In that respect it's a surprise that TEX is still around. On the other hand, humans

also have a tendency to keep things artificially alive and even revive. Can we revive TEX in a few hundred years given the complex code base and Make infrastructure?

4.12 Publishing

What will happen with publishing? In the production notes of some of my recently bought books the author mentions that the first prints were self-published (either or not sponsored). This means that when a publisher "takes over" (which still happens when one scales up) not much work has to be done. Basically the only thing an author needs is a distribution network. My personal experience with for instance CD's produced by a group of musicians is that it is often hard to get it from abroad (if at all) simply because one needs a payment channel and mail costs are also relatively high.

But both demonstrate that given good facilitating options it is unlikely that publishers as we have now have not much change of survival. Add to the argument that while in Gutenbergs time a publisher also was involved in the technology, today nothing innovative comes from publishers: the internet, ebook devices, programs, etc. all come from elsewhere. And I get the impression that even in picking up on technology publishers lag behind and mostly just react. Even arguments like added value in terms of peer review are disappearing with the internet where peer groups can take over that task. Huge amounts of money are wasted on short-term modern media. (I bet similar amounts were never spend on typesetting.)

: Publishers, publishing, publications and their public: as they are now they might not stay around. Lack of long term vision and ideas and decoupling of technology can make sure of that. Publishing will stay but anyone can publish; we only need the infrastructure. Creativity can win over greed and exploitation, small can win over big. And tools like TEX can thrive in there, as it already does on a small scale.

4.13 Understanding

"Why do you use TEX?" If we limit this question to typesetting, you can think of "Why don't you use MS WORD?" "Why don't you use Indesign?", "Why don't you use that macro package?", "Why don't you use this TEX engine?" and alike. I'm sure that most of the readers had to answer questions like this, questions that sort of assume that you're not happy with what you use now, or maybe even suggest that you must be stupid not to use ...

It's not that easy to explain why I use TEX and/or why TEX is good a the job. If you are in a one-to-one (or few) sessions you can demonstrate its virtues but 'selling' it to for instance a publisher is close to impossible because this kind of technology is rather unknown and far from the click-and-point paradigm. It's even harder when students get accustomed to these interactive books from wherein they can even run code snippets although one can wonder how individual these are when a student has the web

as a source of solutions. Only after a long exposure to similar and maybe imperfect alternatives books will get appreciated.

For instance speaking of "automated typesetting" assumes that one knows what typesetting is and also is aware that automated has some benefits. A simple "it's an XML to PDF converter" might work better but that assumes XML being used which for instance not always makes sense. And while hyphenation, fancy font support and proper justification might impress a TEX user it often is less of an argument than one thinks.

The "Why don't you" also can be heard in the TEX community. In the worst case it's accompanied by a "... because everybody uses ..." which of course makes no sense because you can bet that the same user will not fall for that argument when it comes to using an operating system or so. Also from outside the community there is pressure to use something else: one can find defense of minimal markup over TEX markup or even HTML markup as better alternative for dissemination than for instance PDF or TEX sources. The problem here is that old–timers can reflect on how relatively wonderful a current technique really is, given changes over time, but who wants to listen to an old-timer. Progress is needed and stimulating (which doesn't mean that all old technology is obsolete). When I watched Endre eNerd's "The Time Capsule" blu-ray I noticed an Ensoniq Fizmo keyboard and looked up what it was. I ended up in interesting reads where the bottom line was "Either you get it or you don't". Reading the threads rang a bell. As with TEX, you cannot decide after a quick test or even a few hours if you (get the concept and) like it or not: you need days, weeks, or maybe even months, and some actually never really get it after years.

: It is good to wonder why you use some program but what gets used by others depends on understanding. If we can't explain the benefits there is no future for TEX. Or more exact: if it no longer provide benefits, it will just disappear. Just walk around a gallery in a science museum that deals with computers: it can be a bit pathetic experience.

Who knows ...

Stability 5

Introduction

How stable is ConTeXt? This question is hard to answer. For instance MkII hasn't changed for years and seems to work quite well: no changes equals stability. Those who use it can do with what it offers. The potentially sensitive dependencies on for instance fonts are probably absent because there is not much development in the 8 bit fonts arena. As long as these are available we're okay, in fact, OpenType fonts are more a moving target and therefore less stable.

What do we mean by stable? The fundamental differences between an 8 bit engine (and fonts) and an Unicode aware engine able to handle OpenType fonts is substantial which is why we dropped some functionality and added some relevant new. One can consider that a problem but in practice using fonts has become easier so no one is hurt by it. Here we need to keep in mind that pdfTeX is really stable: it uses fonts and technology that doesn't change. On the other hand XeTeX and LuaTeX follow new trends. Thereby XeTeX uses libraries, which introduces a dependency and instability, while LuaTeX assumes solutions in Lua which means that users and macro writers can tweak and thereby also introduce instability (but at least one can adapt that code).

Due to the way the user interface is set up, it is unlikely that ConTeXt will change. But the fact that we now have Lua available means that many commands have been touched. Most behave compatible, some have more functionality, and of course we have a Lua interface. We include a lot of support code which also lessens dependencies.

The user input is normally TeX but when you use xml the move to MkIV meant that we dropped the MkII way of dealing with it in favour of a completely new mechanism. I get the impression that those using xml don't regret that change. Talking of stability the MkIV xml interface is typically a mechanism that is stable and might change little. We can add new trickery but the old stays as it is.

If we look at the output, there is dvi and pdf. In MkII the dvi could become PostScript. As there are different dvi post-processors the backend code was using a plug-in model. Contrary to other macro packages there was only one so called format that could adapt itself to the required (engine specific) output. A ConTeXt run has always been managed by a wrapper so users were not bothered much by what TeX engine they used and/or what backend was triggered. This changed with MkIV where we use just LuaTeX, always produce pdf and optionally can export xml. But again the run is managed by a wrapper, which incidentally is written in Lua and thereby avoids dependencies on for instance Perl, Ruby or Python, which are moving targets, use libraries and additional user code, and thereby are potentially instable too.

The PDF code that is produced is a mix of what the engine spits out and what the macro package injects. The code is normally rather simple. This means that it's no big deal to support the so called standards. It also means that we can support advanced interactivity and other features but these also depends on the viewers used. So, stability here is more fluent, for instance because the PDF standard evolves and/or we need to adapt to viewers. Special demands like tagged PDF have been supported right from the start but how that evolves depends mostly on input from users who need it. Again, that is less important (and crucial) for stability than the rendering capabilities.

The fact that we use LUA creates a dependency on that language but the reason that we use it is *because* it is so stable. We follow the updates and so far that worked out well. Now, say that we had a frozen version of CONTEXT 2010 and LUATEX 1.09 that uses LUA 5.3, would that work? First of all, in 2010 LUATEX itself was evolving so the answer is probably "no", unless one adds a few compatibility patches. I'm not going to try it. The change from 5.1 to 5.2 to 5.3 was not really a problem I think and the few issues could be dealt with easily. If you want long term stability and use a lot of LUA code you can take it into account when coding. Avoiding external libraries is a good start.

Fonts are more than before moving targets. So, if you want stability there you should save them with your document source. The processing of them has evolved and has been improved over time. By now it's rather stable. More recent code can catch more issues and fixes are relatively easy. But it's an area that you always need to check when you update an old distribution. The same is true for language related hyphenation patterns and script specific support. The community is no longer leading in the math department either (OPENTYPE math is a MICROSOFT invention). But, the good news is that the TEX ecosystem is always fast to adapt and can also often provide more functionality.

Vertical spacing, in fact spacing in general is an aera that can always be improved, so there is where you can expect changed. The same is true for side floats or mechanisms where content is somehow attached to other moving content, for instance marginal notes.

But code dealing with fonts, color, scripts, structure, and specific features that once written don't need more, will not change that much. As mentioned for fonts, like any resource, we also depend on third parties. Colors can relate to standards, but their main properties are unchanged. Support for specific scripts can (and will) be improved due to user input and demands so there the users also influence stability. Structure doesn't really influence the overall rendering, but the way you set it up does, but that's user styling. Of course during the transition from MKII to MKIV and the evolution of LUATEX things could be broken, but fixing something structural seldom relates to rendering. If for instance we improve the interpretation of BIBTEX input , which can be real messy, that involves data processing, nor rendering. When we improve support for the APA standard, which is complex, it might involve rendering but then that's asked for and expected. One cannot do better than the input permits.

Publishers

When discussing stability and especially stability as requirement we need to look at the way ConTeXt is used. So let's look at a few scenarios. Say that a publisher gets a camera ready book from an author in PDF format. In that case the author can do all tweaks needed. Now say that the publisher also wants the source code in a format that makes reuse possible.

But let's face reality. Will that publisher really reformat the document in PDF again? It's very unlikely. First of all the original PDF can be kept, and second, a reformat only makes sense after updating the content or going for a completely different layout. It's basically a new book then. In that case literal similarity of output is irrelevant. It is a cheap demand without much substance.

When the source is used for a different purpose the tool used to make the PDF is irrelevant. In that case the coding of the source can matter. If it is in some dialect of TeX, fine, one has to convert it anyway (to suit the other usage). If there is an XML export available, fine too as it can be transformed, given that the structure is rich enough, something that is unlikely to have been checked when the original was archived. Then there could have been the demand for a document in some other format and who can guarantee stability of the tools used there? Just look at how Microsoft Word evolved, or for that matter, its competitors. On the average TeX is more stable as one can snapshot a TeX tree and run binaries for years, if needed, in a virtual machine.

So, I don't think that a publisher is of any relevance in the discussion about stability. Even if we can clearly define what a publisher is, I doubt if publishers themselves can be considered long term stable organizations. Not today. I'm not sure if (especially the large) publishers really deserve a place in the discussion about stability but I'm willing to discuss that when I run into one.

The main problem that an author can face when being confronted with the stability issue this way is that the times are long gone that publishers have a clue about what TeX is, how it evolved and how it always had to and did adapt to changing requirements. If you're lucky you will run into someone who does know all this. They're normally a bit older and have seen the organization from any angles and therefore are fun to work with.

But even then, rendering issues are often not high on their agenda. Outsourcing often has become the modus operandi which basically brings us to the second group involved in this discussion: suppliers.

Suppliers

I don't know many suppliers other than the ones we ran into over a few decades. At least where I live the departments that are responsible for outsourcing typesetting like to deal with only a few large suppliers, interestingly because they assume that they

are stable. However, in my experience hardly any of those seem to have survived. (Of course one can wonder if long term commitment really is that important in a world where companies change so fast.) This is somewhat obscured by the fact that publishers themselves merge, reorganize, move people around, etc. so who can check on the stability of suppliers. It is definitely a fact that at least recently hardly any of them played a rol of any relevance in the development of stable tools. In the past the membership of TEX user groups contained people working at publishers and suppliers but that has changed.

Let's focus on the suppliers that somehow use TEX and let's consider two kind of suppliers: small ones, one were only a few people work, and large ones. The small ones depend on stable TEX distributions, like TEXLive where they can get the resources from: styles, fonts, patterns, binaries. If they get the authors TEX files they need to have that access. They have to rework that input into what the customer demands and that likely involves tweaks. So, maybe they have developed their own additional code. For that code, stability is their own responsibility. Did they tweak core code of a macro package? Fine, but you might have it coming when you update. You cannot expect the evolving free meal world to stick to your commercial needs. A supplier can play safe and somehow involve the developers of macro packages or consult them occasionally, but does that really happen often? Interesting is that a few times that I was asked for input it was also wrapped in obscurity, as if some holy grail of styling was involved, while it's quite likely that the developer of a macro package can write such a style (or extra code) easily and probably also better. There really is not that much unique code around.

Small suppliers can be on mailing lists where they can contribute, get feedback, provide testing, etc. They are part of a process and as such have some influence on stability. If they charge by the page, then a change in their tools can be reflected in what they charge. Basically redoing a book (or so) after a decade is doing a new job. And adapting to some new options in a package, as part of a typesetting job is probably no big deal. Is commercial really more stable than open source free software? Probably not, except from open source software developers whose real objective is to eventually sell their stuff to some company (and cash) and even accept it to be ditched. Small suppliers are more flexible.

The large suppliers are a different group. They often guard their secrets and stay in the dark. They probably seldom share (fundamental) code and information. If they are present in a community it can be for marketing reasons. If at some point a large supplier would demand stability, then my first response would be: sure I can make you a stable setup and maybe even provide intermediate patches but put your money where your mouth is. But that never happened and I've come to the conclusion that we can safely ignore that group. The TEX user groups create distributions and have for instance funded font development and it are the common users who paid for that, not the scale ones. To some extent this is actually good because large (software related) organizations often have special agendas that can contradict what we aim at in the long term.

From the authors perspective there is a dilemma here. When you submit to a publisher who outsources, it can be a demand to deliver in a specific TEX format. Often a PDF comes with the source then, so that the intended rendering is known. Then that source goes to a supplier who then (quite likely) redoes a lot of the coding in some stable subset, maybe even in a very old version of the macro package. If I were such an author I'd render the document in 'as stupid as possible mode' because you gain nothing by spending time on the looks. So, stability within the package that you use is easy and translation from one to another probably also. It's best to check beforehand what will happen with your source and let stability, if mentioned, be their problem. After all they get paid for it.

Suppliers seldom know CONTEXT. An interesting question is if they really know the alternatives well, apart from the bit they use. A well structured CONTEXT source (or probably any source) is often easy to convert to another format. You can assume that a supplier has tools for that (although we're often surprised about the poor quality of tools used). Often the strict demand for some kind of format is an excuse for lack of knowledge. Unfortunately you need a large author base to change that attitude.

Authors

Before we move to some variants of the above, first I will look at stability from the authors perspective. When a book is being written the typesetting more or less happens as part of the process. The way it looks can influence the way you write and vise versa. Once the book is done it can go in print and, unless you were using beta versions of CONTEXT and updated frequently. Normally you will try to work in a stable setup. Of course when a user asks for additional features while working on a project, he or she should also accept other beta features and side effects.

After a few years an author might decide to update the book. The worst that can happen is that the code doesn't run with the latest CONTEXT. This is not so likely because commands are upward compatible. However, the text might come out a bit different, for instance because different fonts or patterns are used. But on the average paragraphs will come out the same in TEX. You can encounter differences in the vertical spacing and page breaks, because that is where improvements are still possible. If you use conceptually and implementation wise complex mechanism like side floats, you can also run into compatibility issues. But all these don't really matter much because the text will be updated anyway and fine-tuning of page breaks (if at all) happens at the end. The more you try to compete with desk top publishing, and the more tweaks you apply, the greater is the risk that you introduce instability. It is okay for a one-time job, but when you come back to it after a decade, be prepared for surprises.

Even if you stick to the original coding, it makes sense to sacrifice some of that stability if new mechanisms have become available. For instance, if you use METAPOST, better ways to solve your problem might have become available. Or if you document is 15 years old, a move from MKII to MKIV is a valid option, in which case you might also consider using the latest fonts.

Of course, when you made a style where you patched core code, you can expect problems, because anything not explicitly mentioned in the interface definition files is subjected to change. But you probably see that coming anyway.

So, is an author (or stand alone user) really dependent on stability? Probably less than thought. In fact, the operating system, internet and browsers, additional tools: all change over time and one adapts. It's something one can live with. Just see how people adapt to phones, tablets, social media, electric cars, etc. As long as the document processes and reasonable output is generated it's fine. And that is always what we aim at! After all we need to be able to use it ourselves, don't we?

Projects

Although it is often overlooked as valid alternative in rendering in large scale projects, ConTeXt is perfect as component in a larger whole. Something goes in, something comes out. In a long term project one can just install a minimal distribution, write styles, and run it for ages. Use a virtual machine and we're talking decades without any change. And, when one updates, it's easy to check if all still works. Often the demands and styles are simple and predictable. It's way more likely that a hard coded solution in some large programming environment has stability issues than that the ConTeXt bit has.

If ConTeXt is used in for instance documentation of (say) software, again there is no real issue. Such documents are simple, evolve and therefore have no stable page flow, and updating ConTeXt is not needed if the once decided upon coding is stable. You don't need the latest features. We've written styles and setups for such tasks and indeed they run for ages.

It can make me smile to see how much effort sometimes goes in low quality rendering where ConTeXt could do a way better job with far less investment in time and money but where using some presumed stable toolkit is used instead, one that comes with expensive licensing, from companies that come and go but shine in marketing. (A valid question is to what extent the quality of and care for documentation reflects the core products that a company produces, at least under the hood.)

The biggest hurdle in setting up a decent efficient workflow is that it has to be seen as a project: proper analysis, proper planning, prototyping and testing, etc. You invest first and gain later. When dealing with paper many publishers still think in price per page and have problems seeing that a stable mostly automated flow in the end can result in a ridiculous low price per page, especially in typesetting on demand.

Hybrids

Last I will mention a setup that we sometimes are involved in. An author writes books and uses TeX. The publisher is okay with that and adds some quality assurance but in the end the product comes from the author. Maybe images are oursourced (not always

for the better) but these can be handled easily. It can be that a copy-editor is involved and that person also then has to use TEX of course, or feedback to the author.

Publishers, and this really depends on knowledgeable persons, which as said can be fun to work with, can look beyond paper and also decide for additional materials, for instance web pages, interactive exercises, etc. In that case either CONTEXT input has to be available as XML (an export) or (often better) XML is the starting point for multiple output. Contrary to what is believed, there are authors out there who have no problem coding in XML directly. They think in structured content and reuse! The fact that they can hit a button in the editor and see the result in PDF helps a lot. It just works.

Here stability is either achieved by simply not updating during a project. There are however cases where an update is needed, for instance because demands changed. An example is a project where ASCIIMATH is used which is a moving target. Of course one can update just that module, and often that works, but not when a module uses some new improved core helpers. Another example is additional proofing options.

The budget of such projects seldom permit patching an existing distribution, so we then just update to the latest but not after checking if the used style works okay. There is no author involvement in this. Depending on the workflow, it can even be that the final rendering which involves fine tuning (side) float placement or page breaks (often educational documents have special demands) is done by us using special directives.

Such hybrid workflows are quite convenient for all parties. The publisher works with the author who likes using these tools, the author can do her or his thing in the preferred way, and we do what we're best in: supporting this. And it scales up pretty well too if needed, without much costs for the publishers.

Conclusion

So what can we conclude with respect to the demand for stability? First of course that it's important that our files keep running well. So, functionality should be stable. Freezing a distribution will make sure that during project you don't run into issues. Many CONTEXT users update frequently in order to benefit from the latest additions. Most will not be harmed by this, but when something really breaks it's users like those on the CONTEXT support list (who often also contribute in helping out other users) that are listened to first. Publishers demands play no role in this, if only because they also play no role in typesetting, and if they want to they should also contribute. The same is true for large suppliers. We're talking of free software often written without any compensation so these parties have no say in the matter unless they pay for it. It's small suppliers, authors and general users that matter most. If CONTEXT is part of a workflow that we support, of course stability is guaranteed quite well, and those paying for that never have an issue with better solutions popping up. In fact, CONTEXT is often just a tool then, one that does the job and questions about stability don't matter much in practice, as long as it does the job well.

The main engine we use, LuaTeX, will be quite stable from version 1.10 and we'll try to make sure that newer versions are capable of running an older ConTeXt, which is easier when no fundamental changes happen in the engine. Maybe a stripped down version of LuaTeX for ConTeXt can facilitate that objective even more.

Users themselves can try to stick to standard ConTeXt features. The more tricks you apply, the less stable your future might be. Most mechanism are not evolving but some, like those that deal with columns, might become better over time. But typesetting in columns is often a one-shot adventure anyway (and who needs columns in the future).

Of one thing users can be sure. There will never be a ConTeXt professional or ConTeXt enterprise. There is only one variant. All users get the same functionality and policies don't change suddenly. There will be no lock in to some cloud or web based service either. Of course one can hire us for support of any kind but that's independent of the distributed package. There is support by users for users on mailing lists and other media. That itself can also guard stability.

But, always keep in mind that stability and progress, either of not driven by the environment that we operate in, can be in conflict.

METATEX, a roadmap 6

6.1 Introduction

Here I will shortly wrap up the state of LUATEX and CONTEXT in fall 2018. I made the first draft of this article as preparation for the CONTEXT meeting where we also discussed the future. I updated the text afterwards to match the decisions made there. It's also a personal summary of thoughts and discussions with team members about where to move next.

6.2 The state of affairs

After a dozen years the development of LUATEX has reached a state where adding more functionality and/or opening up more of the internals makes not much sense. Apart from fixes and maybe some minor extensions, version 1.10 is what you get. Users can do enough in LUA and there is not much to gain in convenience and performance. Of course some of the code can and will be cleaned up, as we still see the effects of going from PASCAL to CWEB to C. In the process consistency is on the radar so we might occasionally add a helper. But we also don't want to move too far away from the original code, which is for instance why we keep names, keys and other properties found in original TEX, which in turn leads to some inconsistencies with extensions added over time. We have to accept that.

Because LUATEX development is closely related to CONTEXT development, especially MKIV, we've also reached the moment that we can get rid of some older code and assume the latest LUATEX to be used. Because we do so much in LUA the question is always to what extent the benefits outweigh the drawbacks. Just in case you wonder why we use LUA extensively, the main reason is that it is easier and more efficient to manage data in this language and modern typesetting needs much data. It also permits us to extend regular TEX functionality. But, one should not overrate the impact: we still let TEX do what TEX is best at!

Performance is quite important. It doesn't make sense to create a powerful typesetting system where processing a page takes a second. We have discussed performance before since one of the complaints about LUATEX is that it is slow. A simple, basic test is this:

```
\starttext
    \dorecurse{1000}{\input tufte \par}
\stoptext
```

This involves 1000 times loading a file (and reporting that on the console, which can influence runtime), typesetting paragraphs, splitting of a page and of course loading

fonts and saving to the PDF file. When I run this on a modest machine, I get these (relative) timings for the (about) 225 pages:

TEX engine used	**PDFTEX**	**LUATEX**	**LUAJITTEX**	**XETEX**
runtime in seconds	2.0	3.9	3.0	8.4

Now, as expected the 8 bit PDFTEX is the winner here but LUATEX is not doing that bad. I don't know why XETEX is so much slower, maybe because its 64 bit binary is less optimal. I once noticed that a 64 bit PDFTEX performed worse on such a test than LUATEX, for which I always use 64 bit binaries.

If you consider that often much more is done than in this example, you can take my word that LUATEX quickly outpaces PDFTEX on more complex tasks. In that sense it is now our benchmark. It must be said that the MKIV code is probably a bit more efficient than the MKII code but that doesn't matter much in this simple test because hardly any macro magic happens here; it mostly tests basic font processing, paragraph building and page construction. I don't think that I can squeeze out more pages per second, at least not without users telling me where they encounter bottlenecks that don't result from their style coding. It's no problem to write inefficient macros (or styles) so normally a user should first carefully check her/his own work. Using a more modern CPU with proper caching and an SSD helps too.

So, to summarize, we can say that with version 1.10 LUATEX is sort of finished. Our mission is now to make LUATEX robust and stable. Things can be added and improved, but these are small and mostly consistency related.

6.3 More in LUA

Till now I always managed to add functionality to CONTEXT without hampering performance too much. Of course the biggest challenge is always in handling fonts and common features like color because that all happens in LUA. So, the question is, what if we delegate more of the core functionality to LUA? I will discuss a few options because the CONTEXT developers and users need to agree on the path to follow. One question there is, are the possible performance hits (which can be an inconvenience) compensated by better and easier typesetting.

Fonts, colors, special typesetting features like spaced kerning, protrusion, expansion, but also dropped caps, line numbering, marginal notes, tables, structure related things, floats and spacing are not open for much discussion. All the things that happen in LUA combined with macros is there and will stay. But how about hyphenation, paragraph building and page building? And how about a leaner and meaner, future safe engine?

Hyphenation is handled in the TEX core. But in CONTEXT already for years one can also use a LUA based variant. There is room for extensions and improvements there. Interesting is that performance is more or less the same, so this is an area where we might switch to the LUA method eventually. It compares to fonts, where node mode is more or less the standard and base mode the old way.

Building the paragraphs in LUA is also available in MKIV, although it needs an update. Again performance is not that bad, so when we add features not possible (or hard to do) in regular TEX, it might actually pay of to default to the par builder written in LUA.

The page builder is also doable in LUA but so far I only played a bit with a LUA based variant. I might pick up that thread. However, when we would switch to LUA there, it might have a bit of a penalty, unless we combine it with some other mechanisms which is not entirely trivial, as it would mean a diversion from the way TEX does it normally.

How about math? We could at some point do math rendering in LUA but because the core mechanism is the standard, it doesn't really makes much sense. It would also touch the soul of TEX. But, I might give it a try, just for fun, so that I can play with it a bit. It's typically something for cold and rainy days with some music in the background.

We already use LUA in the frontend: locating and reading files in TEX, XML, LUA and whatever input format. Normalization and manipulation is all active and available. The backend is also depending on LUA, like support for special PDF features and exporting to XML. The engine still handles the page stream conversion, font inclusion and object management.

The inclusion of images is also handled by the engine, although in CONTEXT we can delegate PDF inclusion to LUA. Interesting is that this has no performance hit.

With some juggling the page stream conversion can also be done in LUA, and I might move that code into the CONTEXT distribution. Here we do have a performance hit: about one second more runtime on the 14 seconds needed for the 300 page LUATEX manual and just over more than half a second on a 11 second LUAJITTEX run. The manual has lots of tables, verbatim, indices and uses color as well as a more than average number of fonts and much time is spent in LUA. So there is a price to pay there. I tried to speed that up but there is not much to gain there.

So, say that we default to LUA based hyphenation, which enables some new functionality, LUA based par building, which permits some heuristics for corner cases, and LUA based page building, which might result in more control over tricky cases. A total performance hit of some 5% is probably acceptable, especially because by that time I might have replaced my laptop and won't notice the degrade. This still fits in the normal progress and doesn't really demand a roadmap or wider acceptance. And of course we would still use the same strategies as implemented in traditional TEX as default anyway.

6.4 A more drastic move

More fundamental is the question whether we delegate more backend activity to LUA code. If we decide to handle the page stream in LUA, then the next question is, why not also delegate object management and font inclusion to LUA. Now, keep in mind that this is all very CONTEXT specific! Already for more than a decade we delegate a lot to

LUA, and also we have a rather tight control over this core functionality. This would mean that CONTEXT doesn't really need the backend code in the engine.[4]

That situation is actually not unique. For instance, already for a while we don't need the LUATEX font loader either, as loading the OPENTYPE files is done in LUA. So, we could also get rid of the font loader code. Currently some code is shared with the font inclusion in the backend but that can be isolated.

You can see a TEX engine as being made from several parts, but the core really concerns only two processes: reading, storing and expanding macros on the one hand, and converting a stream of characters into lines, paragraphs, pages etc. Fonts are mostly an abstraction: they are visible in so called glyph nodes as font identifier (a number) and character code (also a number) properties. The result, nowadays being PDF, is also an abstraction: at some point the engine converts the to be shipped out box in PDF instructions, and in our case, relatively simple ones. The backend registers which characters and fonts are used and also includes the right resources. But, the backend is not part of the core as such! It has been introduced in PDFTEX and is a so called extension.

So, what does that all mean for a future version of CONTEXT and LUATEX? It means that we can decide to follow up with a CONTEXT that does more in LUA, which means not hard coded in a binary, on the one hand, but that we can also decide to strip the engine from non-core code. But, given that LUATEX is also used in other macro packages, this would mean a different engine. We cannot say that LUATEX is stable when we also experiment with core components.

We've seen folks picking up experimental versions assuming that it is a precursor to official code. So, in order to move on we need to avoid confusion: we need to use another name. Choosing a name is always tricky but as Taco already registered the METATEX domain, and because in the CONTEXT distribution you will find references to METATEX, we will use that name for the future engine. Adding LUA to that name makes sense but then the name would become too long.

The main difference between METATEX and LUATEX would be that the former has no file lookup library, no hardcoded font loader, and no backend generator (but possibly some helpers, and these need time to evolve). We're basically back where TEX started but instead of coding these extensions in PASCAL or C we use LUA. We're also kind of back to when we first started experimenting with LUATEX in CONTEXT where test, write and rewrite were going in parallel. But, as said, we cannot impose that on a wide audience.

If we go for such a lean and mean follow up, then we can also do a more drastic cleanup of obsolete code in CONTEXT (dating from ε-TEX, PDFTEX, ALEPH, etc.). We then are sort of back to where it all started: we go back to the basics. This might mean dropping some primitives (one can define them as dummy). Of course we could generalize some of the

[4] For generic packages like TikZ we (can) provide some primitive emulators, which is rather trivial to implement.

CONTEXT code to provide the kicked out functionality but would that pay of? Probably not.

Just for the record: replacing the handling of macros, registers, grouping, etc. to LUA is not really an option as the performance hit would make a large system like CONTEXT sort of unusable: it's no option and not even considered (although I must admit that I have some experimental LUA based TEX parser code around).

It is quite likely that building METATEX from source for the moment will be an option to the build script. But we can also decide to simplify that process, which is possible because we only need one binary. But in general we can assume that one can generate METATEX and LUATEX from the same source. A first step probably is a further isolation of the backend code. The fontloader and file handling code already can be made optional.

Given that we only need one binary (it being LUATEX or METATEX) and nowadays only use OPENTYPE fonts, one can even start thinking of a mini distribution, possibly with a zipped resource tree, something we experimented with in the early days of LUATEX.

Another though I have been playing with is a better separation between low level and high level CONTEXT commands, and whether the low level layer should be more generic in nature (so that one can run specific packages on top of it instead of the whole of CONTEXT) but that might not be worth the trouble.

6.5 Interlude

If we look at the future, it's good to also look at the past. Opening up TEX the way we did has many advantages but also potential drawbacks. It works quite well in CONTEXT because we ship an integrated package. I don't think that there are many users who kick in their own callbacks. It is possible but completely up to the user to make sure things work out well. Performance hits, interference, crashes: those who interfere with the internals can sort that out themselves. I'm not sure how well that works out in other macro packages but it is a time bomb if users start doing that. Of course the documented interfaces to use LUA in CONTEXT are supported. So far I think we're not yet bitten in the tail. We keep this aspect out of the discussion.

Another important aspect is stability of the engine. Sometimes we get suggestions for changes or patches that works for a specific case but for sure will have side effects on CONTEXT. Just as we don't test LATEX side effects, LATEX users don't check CONTEXT. And we're not even talking of users who expect their code to keep working. A tight control over the source is important but cannot be we will not be around for ever. This means that at some point LUATEX should not be changed any more, even when we observe side effects we want to get rid of, because these side effects can be in use. This is another argument for a stripped down engine. The less there is to mess with, the less the mess.

6.6 Audience

So how about ConTeXt itself? Of course we can make it better. We can add more examples and more documentation. We can try to improve support. The main question for us (as developers) is who actually is our audience. From the mails coming to the ConTeXt support list it looks like a rather diverse group of users.

At TeX meetings there are often discussions about promoting TeX. I can agree on the fact that even for simple documents it makes a lot of sense to use TeX, but who will take the first hurdles? How many people really produce a lot of documents? And how many need TeX after maybe a short period of (enforced) usage at the university?

It's not trivial to recognize the possibilities and power of the LuaTeX-ConTeXt combination. We never got any serious requests for support from large organizations. In fact, we do use this combination in a few projects for educational publishers, but there it's actually the authors and editors doing the work. It's seldom company policy to use tools that efficiently automate typesetting. I dare to say that publishers are not really an audience at all: they normally delegate the task. They might accept TeX documents but let them rekey or adapt far-far-away and as cheap as possible. Thinking of it, the main reason for Don Knuth for writing TeX in the first place was the ability to control the look and feel and quality. It were developments at typesetters and publishers that triggered development of TeX. It was user demand. And the success of TeX was largely due to the unique personality and competence of the author.

System integrators qualify as audience but I fear that TeX is not considered hip and modern. It doesn't seem to matter if you can demonstrate that it can do a wonderful job efficiently and relatively cheap. Also the fact that an installation can be very stable on the long run is of no importance. Maybe that audience (market place) is all about "The more we have to program and update regularly, the merrier.". Marketing TeX is difficult.

Those who render multiple products, maintain manuals, have to render many documents automatically qualify as audience. But often company policies, preferred suppliers, so called standard tools etc. are used as argument against TeX. It's a missed opportunity.

One needs a certain mindset to recognize the potential and the question is, how do we reach that audience. Drawing a roadmap for that is not easy but worth discussing. We're open for suggestions.

6.7 Conclusion

At the ConTeXt user meeting those present agreed that moving forward this way makes sense. This means that we will explore a lean and mean MetaTeX alongside LuaTeX. There is no rush and it's all volunteer work so we will take our time for this. It boils down to some reshuffling of code so that we can remove the built-in font loader, file

handling, and probably also SYNCTEX because we can emulate that. Then the backend with its font inclusion code will be cleaned up a bit (we even discussed only supporting modern wide fonts). It's no big deal to adapt CONTEXT to this (so it can and will support both LUATEX and METATEX). Eventually the backend might go away but now we're talking years ahead. By then we can also explore the option to make METATEX start out as a LUA function call (the main control loop) and become reentrant. There will probably not be many changes to the opened up TEX kernel, but we might extend the METAPOST part a bit (some of that was discussed at the meeting) especially because it is a nice tool to visualize big data.

As with LUATEX development we will go in small steps so that we keep a working system. Of course LUATEX is always there as stable fallback. The experiments will mostly happen in the experimental branch and binaries will be generated using the compile farm on the CONTEXT garden, just as happens now. This also limits testing and exploring to the CONTEXT community so that there are no side effects for mainstream LUATEX usage.

Nowadays, instead if roadmaps, we tend to use navigational gadgets that adapt themselves to the situation. On the road by car this can mean a detour and when walking around it can be going to suggested points of interest. During the excursion at the meeting, we noticed that after the drivers (navigators) synchronized their gadget with Jano, the routes that were followed differed a bit. We saw cars in front of going a different direction and cars behind us arriving from a different direction. So, even when we talk about roadmaps, our route can be adapted to the situation.

Now here is something to think about. If you look at the TEX community you will notice that it's an aging community. User groups seem to loose members, although the CONTEXT group is currently still growing. Fortunately we see a new generation taking interest and the CONTEXT users are a pleasant mix and it makes me stay around. I see it as an 'old timers' responsibility to have TEX and its environment in a healthy state by the time I retire from it (although I have no plans in that direction). In parallel to the upcoming development I think we will also see a change in TEX use and usage. This aspect was also discussed at the meeting and for sure will get a follow up on the mailing lists and future meetings. It might as well influence the decisions we make the upcoming years. So far TEX has never failed us in it's flexibility and capacity to adapt, so let's end on that positive note.

About what CONTEXT isn't 7

7.1 Introduction

It really puzzles me why, when someone someplace asks if CONTEXT is suitable for her or is his needs, there are answers like: "You need to think of CONTEXT as being kind of plain TEX: you have to define everything yourself." That answer probably stems from the fact that for LATEX you load some style that defines a lot, which you then might need to undefine or redefine, but that's not part of the answer.

In the following sections I will go into a bit more detail of what plain TEX is and how it influences macro packages, especially CONTEXT. I'm sure I have discussed this before so consider this another go at it.

The `plain.tex` file start with the line:

```
% This is the plain TeX format that's described in The TeXbook.
```

A few lines later we read:

```
% And don't modify the file under any circumstances.
```

So, this format related to the TEX reference. It serves as a template for what is called a macro package. Here I will not go into the details of macro programming but an occasional snippet of code can be illustrative.

7.2 Getting started

The first code we see in the plain file is:

```
\catcode`\{=1 % left brace is begin-group character
\catcode`\}=2 % right brace is end-group character
\catcode`\$=3 % dollar sign is math shift
\catcode`\&=4 % ampersand is alignment tab
\catcode`\#=6 % hash mark is macro parameter character
\catcode`\^=7 \catcode`\^^K=7 % circumflex and uparrow are for superscripts
\catcode`\_=8 \catcode`\^^A=8 % underline and downarrow are for subscripts
\catcode`\^^I=10                          % ascii tab is a blank space
\chardef\active=13    \catcode`\~=\active % tilde is active
\catcode`\^^L=\active \outer\def^^L{\par} % ascii form-feed is "\outer\par"
```

Assigning catcodes to the braces and hash are needed in order to make it possible to define macros. The dollar is set to enter math mode and the ampersand becomes a separator in tables. The superscript and subscript also relate to math. Nothing demands

these bindings but they are widely accepted. In this respect ConTeXt is indeed like plain.

The tab is made equivalent to a space and a tilde is made active which means that later on we need to give it some meaning. It is quite normal to make that an unbreakable space, and one with the width of a digit when we're doing tables. Now, nothing demands that we have to assume ASCII input but for practical reasons the formfeed character is made equivalent to a `\par`.

Now what do these `^^K` and similar triplets represent? The `^^A` represents character zero and normally all these control characters below decimal 32 (space) are special. The `^^I` is the ASCII tab character, and `^^L` the formfeed. But, the ones referred to as uparrow and downarrow in the comments have only meaning on certain keyboards. So these are typical definitions that only made sense for Don Knuth at that time and are not relevant in other macro packages that aim at standardized input media.

```
% We had to define the \catcodes right away, before the message line, since
% \message uses the { and } characters. When INITEX (the TeX initializer) starts
% up, it has defined the following \catcode values:
%
% \catcode`\^^@=9  % ascii null is ignored
% \catcode`\^^M=5  % ascii return is end-line
% \catcode`\\=0    % backslash is TeX escape character
% \catcode`\%=14   % percent sign is comment character
% \catcode`\ =10   % ascii space is blank space
% \catcode`\^^?=15 % ascii delete is invalid
% \catcode`\A=11 ... \catcode`\Z=11 % uppercase letters
% \catcode`\a=11 ... \catcode`\z=11 % lowercase letters
% all others are type 12 (other)
```

The comments above speak for themselves. Changing catcodes is one way to adapt interpretation. For instance, in verbatim mode most catcodes can best be made letter or other. In ConTeXt we always had so called catcode regimes: for defining macros, for normal text, for XML, for verbatim, etc. In MkIV this mechanism was adapted to the new catcode table mechanism available in that engine. It was one of the first things we added to LuaTeX. So, again, although we follow some standards (expectations) ConTeXt differs from plain.

```
% We make @ signs act like letters, temporarily, to avoid conflict between user
% names and internal control sequences of plain format.

\catcode`@=11
```

In ConTeXt we went a step further and when defining macros also adapted the catcode of `!` and `?` and later in MkIV `_`. When we're in unprotected mode this applies. In addition to regular text input math is dealt with:

```
% INITEX sets up \mathcode x=x, for x=0..255, except that
%
% \mathcode x=x+"7100, for x = `A to `Z and `a to `z;
% \mathcode x=x+"7000, for x = `0 to `9.

% The following changes define internal codes as recommended in Appendix C of
% The TeXbook:

\mathcode`\^^@="2201 % \cdot
\mathcode`\^^A="3223 % \downarrow
\mathcode`\^^B="010B % \alpha
\mathcode`\^^C="010C % \beta
.....................
\mathcode`\|="026A
\mathcode`\}="5267
\mathcode`\^^?="1273 % \smallint
```

Here we see another set of definitions but the alphabetic ones are not defined in ConTeXt, they are again bindings to the authors special keyboard.

```
% INITEX sets \sfcode x=1000 for all x, except that \sfcode`X=999 for uppercase
% letters. The following changes are needed:

\sfcode`\)=0 \sfcode`\'=0 \sfcode`\]=0

% The \nonfrenchspacing macro will make further changes to \sfcode values.
```

Definitions like this depend on the language. Because original TeX was mostly meant for typesetting English, these things are hard coded. In ConTeXt such definitions relate to languages.

I show these definitions because they also illustrate what TeX is about: typesetting math:

```
% Finally, INITEX sets all \delcode values to -1, except \delcode`.=0

\delcode`\(="028300
\delcode`\)="029301
\delcode`\[="05B302
\delcode`\]="05D303
\delcode`\<="26830A
\delcode`\>="26930B
\delcode`\/="02F30E
\delcode`\|="26A30C
\delcode`\\="26E30F

% N.B. { and } should NOT get delcodes; otherwise parameter grouping fails!
```

Watch the last comment. One of the complications of TEX is that because some characters have special meanings, we also need to deal with exceptions. It also means that arbitrary input is not possible. For instance, unless the percent character is made a letter, everything following it till the end of a line will be discarded. This is an areas where macro packages can differ but in MKII we followed these rules. In MKIV we made what we called `\nonknuthmode` default which means that ampersands are just that and scripts are only special in math (there was also `\donknuthmode`). So, CONTEXT is not like plain there.

7.3 Housekeeping

The next section defines some numeric shortcuts. Here the fact is used that a defined symbolic character can act as counter value. When the number is larger than 255 a math character is to be used. In LUATEX, which is a UNICODE engine character codes can be much larger.

```
% To make the plain macros more efficient in time and space, several constant
% values are declared here as control sequences. If they were changed, anything
% could happen; so they are private symbols.

\chardef\@ne=1
\chardef\tw@=2
\chardef\thr@@=3
\chardef\sixt@@n=16
\chardef\@cclv=255
\mathchardef\@cclvi=256
\mathchardef\@m=1000
\mathchardef\@M=10000
\mathchardef\@MM=20000
```

In CONTEXT we still support these shortcuts but never use them ourselves. We have plenty more variables and constants and nowadays always use verbose names. (There was indeed a time when each extra characters depleted string memory more and more so then using short command names made sense.) The comment is right that using such variables is more efficient, for instance once loaded a macro is a sequence of tokens, so `\@one` takes one memory slot. In the case of the first three the saving is zero and even interpreting a single character token `3` is not less efficient than `\thr@@`, but in the case of `\@cclv` the three tokens `255` take more memory and also trigger the number scanner which is much slower than simply taking the meaning of the `\chardef`'d token. However, the CONTEXT variable `\plusone` is as efficient as the `\@ne` and it looks prettier in code too (and I'm very sensitive for that). So, here CONTEXT is definitely different!

It makes no sense to show the next section here: it deals with managing registers, like counters and dimensions and token lists. Traditional TEX has 255 registers per category. Associating a control sequence (name) with a specific counter is done with `\countdef` but I don't think that you will find a macro package that expects a user to use that

primitive. Instead it will provide a \newcount macro. So yes, here ConTeXt is like plain.

Understanding these macros is a test case for understanding TeX. Take the following snippet:

```
\let\newtoks=\relax % we do this to allow plain.tex to be read in twice
\outer\def\newhelp#1#2{\newtoks#1#1\expandafter{\csname#2\endcsname}}
\outer\def\newtoks{\alloc@5\toks\toksdef\@cclvi}
```

The \outer prefix flags macros as to be used at the outermost level and because the \newtoks is in the macro body of \newtoks it has to be relaxed first. Don't worry if you don't get it. In ConTeXt we have no outer macros so the definitions differ there.

The plain format assumes that the first 10 registers are used for scratch purposes, so best also assume this to be the case in other macro packages. There is no need for ConTeXt to differ from plain here. The definitions of box registers and inserts are special: there is no \boxdef and inserts use multiple registers. Especially the allocation of inserts is macro package specific. Anyway, ConTeXt users never see such details because inserts are used as building blocks deep down.

Right after defining the allocators some more constants are defined:

```
% Here are some examples of allocation.

\newdimen\maxdimen \maxdimen=16383.99999pt % the largest legal <dimen>
```

We do have that one, as it's again a standard but we do have more such constants. This definition is kind of interesting as it assumes knowledge about what is acceptable for TeX as dimension:

```
{\maxdimen=16383.99999pt \the\maxdimen \quad \number\maxdimen}
{\maxdimen=16383.99998pt \the\maxdimen \quad \number\maxdimen}
```

16383.99998pt 1073741823
16383.99998pt 1073741823

Indeed it is the largest legal dimension but the real largest one is slighly less. We could also have said the following, which also indicates what the maximum cardinal is:

```
\newdimen\maxdimen \maxdimen=1073741823sp
```

We dropped some of the others defined in plain. So, ConTeXt is a bit like plain but differs substantially. In fact, MkII already used a different allocator implementation and MkIV is even more different. We also have more \new things.

The \newif definition also differs. Now that definition is quite special in plain TeX, so if you want a challenge, look it up. It defines three macros as the comment says:

```
% For example, \newif\iffoo creates \footrue, \foofalse to go with \iffoo.
```

The `\iffoo` is either equivalent to `\iftrue` or `\iffalse` because that is what TEX needs to see in order to be able to skip nested conditional branches. In CONTEXT we have so called conditionals, which are more efficient. So, yes, you will find such defined ifs in the CONTEXT source but way less than you'd expect in such a large macro package: CONTEXT code doesn't look much like plain code I fear.

7.4 Parameters

A next stage sets the internal parameters:

```
% All of TeX's numeric parameters are listed here, but the code is commented out
% if no special value needs to be set. INITEX makes all parameters zero except
% where noted.
```

We use different values for many of them. The reason is that the plain TEX format is set up for a 10 point Computer Modern font system, and for a particular kind of layout, so we use different values for:

```
\hsize=6.5in
\vsize=8.9in
\maxdepth=4pt
```

and

```
\abovedisplayskip=12pt plus 3pt minus 9pt
\abovedisplayshortskip=0pt plus 3pt
\belowdisplayskip=12pt plus 3pt minus 9pt
\belowdisplayshortskip=7pt plus 3pt minus 4pt
```

No, here CONTEXT is not like plain. But, there is one aspect that we do inherit and that is the ratio. Here a 10 point relates to 12 point and this 1.2 factor is carried over in some defaults in CONTEXT. So, in the end we're a bit like plain.

After setting up the internal quantities plain does this:

```
\newskip\smallskipamount \smallskipamount=3pt plus 1pt minus 1pt
\newskip\medskipamount \medskipamount=6pt plus 2pt minus 2pt
\newskip\bigskipamount \bigskipamount=12pt plus 4pt minus 4pt
\newskip\normalbaselineskip \normalbaselineskip=12pt
\newskip\normallineskip \normallineskip=1pt
\newdimen\normallineskiplimit \normallineskiplimit=0pt
\newdimen\jot \jot=3pt
\newcount\interdisplaylinepenalty \interdisplaylinepenalty=100
\newcount\interfootnotelinepenalty \interfootnotelinepenalty=100
```

The first three as well as the following three related variables are not internal quantities but preallocated registers. These are not used in the engine but in macros. In ConTeXt we do provide them but the first three are never used that way. The last three are not defined at all. So, ConTeXt provides a bit what plain provides, just in case.

7.5 Fonts

The font section is quite interesting. I assume that one reason why some want to warn users against using ConTeXt is because it supports some of the font switching commands found in plain. We had no reasons to come up with different ones but they do different things anyway, for instance adapting to situations. So, in ConTeXt you will not find the plain definitions:

```
\font\tenrm=cmr10 % roman text
\font\preloaded=cmr9
\font\preloaded=cmr8
\font\sevenrm=cmr7
\font\preloaded=cmr6
\font\fiverm=cmr5
```

There is another thing going on here. Some fonts are defined `\preloaded`. So, `cmr9` is defined, and then `cmr8` and `cmr6`. But they all use the same name. Later on we see:

```
\let\preloaded=\undefined % preloaded fonts must be declared anew later.
```

If you never ran into the relevant part of the TeX book or read the program source of TeX, you won't realize that preloading means that it stays in memory which in turn means that when it gets (re)defined later, the font data doesn't come from disk. In fact, as the plain format is normally dumped for faster reload later on, the font data is also retained. So, preloading is a speed up hack. In ConTeXt font loading has always been delayed till the moment a font is really used. This permits plenty of definitions and gives less memory usage. Of course we do reuse fonts once loaded. All this, plus the fact that we have a a system of related sizes, collections of families, support multiple font encodings alongside, collect definitions in so called typescript, etc. makes that the ConTeXt font subsystem is far from what plain provides. Only some of the command stick, like `\rm` and `\bf`.

The same is true for math fonts, where we can have different math font setups in one document. Definitely in MkII times, we also had to work around limitations in the number of available math families, which again complicated the code. In MkIV things are even more different, one can even consider the implementation somewhat alien for a standard macro package, but that's for another article (if at all).

7.6 Macros

Of course ConTeXt comes with macros, but these are organized in setups, environments, instances, etc. The whole process and setup is keyword driven. Out of the box

all things work: nothing needs to be loaded. If you want it different, you change some settings, but you don't need to load something. Maybe that last aspect is what is meant with ConTeXt being like plain: you don't (normally) load extra stuff. You just adapt the system to your needs. So there we proudly follow up on plain TeX.

In the plain macro section we find definitions like:

```
\def\frenchspacing{\sfcode`\.\@m \sfcode`\?\@m \sfcode`\!\@m
  \sfcode`\:\@m \sfcode`\;\@m \sfcode`\,\@m}
\def\nonfrenchspacing{\sfcode`\.3000\sfcode`\?3000\sfcode`\!3000%
  \sfcode`\:2000\sfcode`\;1500\sfcode`\,1250 }
```

and:

```
\def\space{ }
\def\empty{}
\def\null{\hbox{}}

\let\bgroup={
\let\egroup=}
```

and:

```
\def\nointerlineskip{\prevdepth-1000\p@}
\def\offinterlineskip{\baselineskip-1000\p@
  \lineskip\z@ \lineskiplimit\maxdimen}
```

Indeed we also provide these, but apart from the two grouping related aliases their implementation is different in ConTeXt. There is no need to reinvent names.

For a while we kept (and did in MkII) some of the plain helper macros, for instance those that deal with tabs, but we have several more extensive table models that are normally used. We always had our own code for float placement, and we also have more options there. Footnotes are supported but again we have multiple classes, placements, options, etc. Idem for itemized lists, one of the oldest mechanisms in ConTeXt. We don't have `\beginsection` but of course we do have sectioning commands, and have no `\proclaim` but provide lots of descriptive alternatives, so many that I forgot about most of them by now (so plain is a winner in terms of knowing a macro package inside out).

The fact that we use tables, floats and footnotes indeed makes ConTeXt to act like plain, but that's then also true for other macro packages. A fact is that plain sets the standard for how to think about these matters! The same is true for naming characters:

```
\chardef\%=`\%
\chardef\&=`\&
\chardef\#=`\#
\chardef\$=`\$
```

```
\chardef\ss="19
\chardef\ae="1A
\chardef\oe="1B
\chardef\o="1C
\chardef\AE="1D
\chardef\OE="1E
\chardef\O="1F
\chardef\i="10 \chardef\j="11 % dotless letters
```

But we have many more and understandable the numbers are different in ConTeXt because we use different font (encodings). Their implementation is more adaptive. The same is true for accented characters:

```
\def\`#1{{\accent18 #1}}
\def\'#1{{\accent19 #1}}
```

The definitions in MkII are different (in most cases we use native glyphs) and in MkIV we use Unicode anyway. I think that the `\accent` command is only used in a few exceptional cases (like very limited fonts) in MkII and never in MkIV. The implementation of for instance accents (and other pasted together symbols) in math is also quite different.

There are also definitions that seem to be commonly used in macro packages but that we never use in ConTeXt because they interfere badly with all kind of other mechanisms, so you will find no usage of

```
\def\leavevmode{\unhbox\voidb@x} % begins a paragraph, if necessary
```

in ConTeXt. In order to stress that we provide `\dontleavehmode`, a wink to not using the one above.

The macro section ends with lots of math definitions. Most of the names used are kind of standard so again here ConTeXt is like plain, but the implementation can differ as does the level of control.

7.7 Output

Once a page is ready it gets wrapped up and shipped out. Here ConTeXt is very different from plain. The amount of code in plain is not that large but the possibilities aren't either, which is exactly what the objectives demand: a simple (example) format that can be described in the TeXbook. But, as with other aspects of plain, it steered the way macro packages started out as it showed the way. As did many examples in the TeX book.

7.8 Hyphenation

As an afterthought, the plain format ends with loading hyphenation patterns, that is the English ones. That said it will be clear that ConTeXt is not like plain: we support

many languages, and the subsystem deals with labels, specific typesetting properties, etc. too.

```
\lefthyphenmin=2 \righthyphenmin=3 % disallow x- or -xx breaks
\input hyphen
```

We don't even use these patterns as we switched to UTF long ago (also in MKII) if only because we had to deal with a mix of font encodings. But we did preload the lot there. In MKIV again things are much different.

7.9 Conclusion

The plain format does (and provides) what it is supposed to do. It is a showcase of possibilities and part of the specification. In that respect it's nice that CONTEXT is considered to be like plain. But if it wasn't more, there was no reason for its existence. Like more assumptions about CONTEXT it demonstrates that those coming up with answers and remarks like that probably missed something in assessing CONTEXT. Just let users find out themselves what suits best (and for some that actually might be plain TEX).